

Sheep_Cat_Elephant

May 25, 2026

1 Construire un réseau de convolutions (CNN) pour tester trois classes

1.1 Description

Ce notebook entraîne un CNN pour une tâche de **classification à 3 classes** : mouton, chat et éléphant. Il explore successivement plusieurs stratégies d'amélioration face au surapprentissage : réglage du modèle (régularisation L2, dropout), augmentation de données via `ImageDataGenerator`, et **transfer learning** avec ResNet50.

```
[18]: import os
from os import listdir
from os.path import isfile, join
import numpy as np
import pandas as pd
import random
import zipfile

import matplotlib.pyplot as plt
import seaborn as sns
import cv2

from sklearn.datasets import make_moons
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline

import joblib

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import Input, Model, regularizers
from tensorflow.keras.optimizers import Adam, SGD
```

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ␣
    ↪ ReduceLROnPlateau
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, ␣
    ↪ Dropout, BatchNormalization, GlobalAveragePooling2D
from tensorflow.keras.regularizers import l2
from keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator #␣
    ↪ Générateur d'images pour l'augmentation des données
from tensorflow.keras.applications.resnet50 import ResNet50 , preprocess_input

# Pour définir la graine tout de suite
np.random.seed(42)
tf.random.set_seed(42)

print(tf.__version__)

```

2.18.0

```

[2]: # Cellule générée par ChatGPT

def plot_history_simple(history):
    """
    Trace côte à côte les courbes Loss et Accuracy (train/val si dispo)
    à partir d'un objet Keras History.
    """
    hist = history.history

    # compatibilité anciennes versions ("acc"/"val_acc")
    acc_key = "accuracy" if "accuracy" in hist else "acc"
    val_acc_key = "val_accuracy" if "val_accuracy" in hist else "val_acc"

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    # --- Loss ---
    if "loss" in hist:
        axes[0].plot(hist["loss"], label="train")
    if "val_loss" in hist:
        axes[0].plot(hist["val_loss"], label="val")
    axes[0].set_title("Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Loss")
    axes[0].legend()

    # --- Accuracy ---
    if acc_key in hist:
        axes[1].plot(hist[acc_key], label="train")
    if val_acc_key in hist:

```

```

        axes[1].plot(hist[val_acc_key], label="val")
        axes[1].set_title("Accuracy")
        axes[1].set_xlabel("Epoch")
        axes[1].set_ylabel("Accuracy")
        axes[1].legend()
    else:
        axes[1].set_visible(False) # si pas d'accuracy, on masque le 2e plot

plt.tight_layout()
plt.show()

def plot_confusion_matrix_simple(y_true, y_pred, labels=(0,1),
                                normalize=None, title="Confusion matrix"):
    """
    Affiche une matrice de confusion.
    Args:
        y_true, y_pred : vecteurs de labels
        labels          : ordre des classes à afficher
        normalize       : None, 'true', 'pred', ou 'all' (comme scikit-learn)
        title           : titre de la figure
    """
    cm = confusion_matrix(y_true, y_pred, labels=labels, normalize=normalize)
    fig, ax = plt.subplots(1, 1, figsize=(4, 4))
    im = ax.imshow(cm, interpolation='nearest')
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(len(labels)),
        yticks=np.arange(len(labels)),
        xticklabels=labels,
        yticklabels=labels,
        xlabel="Predicted label",
        ylabel="True label",
        title=title if normalize is None else f"{title} (normalized: {normalize})"
    )

    # annotations
    thresh = cm.max() / 2.0 if cm.size else 0.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            txt = f"{cm[i, j]:.2f}" if normalize else f"{int(cm[i, j])}"
            ax.text(j, i, txt,
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black")

```

```

plt.tight_layout()
plt.show()

def plot_history_cv_mean_std(all_histories, alpha=0.2):
    """
    all_histories : list[pd.DataFrame]
        Chaque DF doit contenir :
        → ['accuracy', 'loss', 'val_accuracy', 'val_loss', 'epoch', 'fold'].
        (c'est exactement ton format actuel)

    Trace loss et accuracy (train/val) avec moyenne ± écart-type par epoch.
    Si certains folds s'arrêtent plus tôt (early stopping), la moyenne/STD
    sur les dernières époques est calculée sur les folds encore présents.
    """
    import pandas as pd
    import matplotlib.pyplot as plt

    # 1) Concat & agrégation
    df = pd.concat(all_histories, ignore_index=True)
    cols = [c for c in ["loss", "val_loss", "accuracy", "val_accuracy"] if c in df.columns]
    stats = df.groupby("epoch")[cols].agg(["mean", "std"])
    stats.columns = [f"{m}_{s}" for m, s in stats.columns] # ex: loss_mean
    → ('loss', 'mean') -> 'loss_mean'
    stats = stats.reset_index()
    epochs = stats["epoch"].values

    # 2) Plot côte à côte
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    # --- Loss ---
    if "loss_mean" in stats:
        axes[0].plot(epochs, stats["loss_mean"], label="train")
        if "loss_std" in stats:
            axes[0].fill_between(epochs, stats["loss_mean"]-stats["loss_std"],
                                stats["loss_mean"]+stats["loss_std"],
                                → alpha=alpha)
        if "val_loss_mean" in stats:
            axes[0].plot(epochs, stats["val_loss_mean"], label="val")
            if "val_loss_std" in stats:
                axes[0].fill_between(epochs,
                                    → stats["val_loss_mean"]-stats["val_loss_std"],
                                    → stats["val_loss_mean"]+stats["val_loss_std"], alpha=alpha)
            axes[0].set_title("Loss"); axes[0].set_xlabel("Epoch"); axes[0].
            → set_ylabel("Loss"); axes[0].legend()

```

```

# --- Accuracy ---
if "accuracy_mean" in stats:
    axes[1].plot(epochs, stats["accuracy_mean"], label="train")
    if "accuracy_std" in stats:
        axes[1].fill_between(epochs,
↪stats["accuracy_mean"]-stats["accuracy_std"],
                                ↪
↪stats["accuracy_mean"]+stats["accuracy_std"], alpha=alpha)
    if "val_accuracy_mean" in stats:
        axes[1].plot(epochs, stats["val_accuracy_mean"], label="val")
        if "val_accuracy_std" in stats:
            axes[1].fill_between(epochs,
↪stats["val_accuracy_mean"]-stats["val_accuracy_std"],
                                    ↪
↪stats["val_accuracy_mean"]+stats["val_accuracy_std"], alpha=alpha)
        axes[1].set_title("Accuracy"); axes[1].set_xlabel("Epoch"); axes[1].
↪set_ylabel("Accuracy"); axes[1].legend()

plt.tight_layout()
plt.show()

def plot_decision_boundary(pipe, X, y=None, title=None, h=0.02, padding=0.5,
↪threshold=0.5, ax=None):
    """
    Affiche la frontière de décision d'un modèle 2D (ex: make_moons).
    pipe : Pipeline déjà fit (ex: StandardScaler + KerasClassifier)
    X, y : données à superposer (y optionnel pour juste la frontière)
    h : pas de la grille, padding : marge autour des données
    threshold : seuil de classification si proba disponible
    """
    if ax is None:
        fig, ax = plt.subplots(figsize=(6,5))

    # -- Grille 2D
    x_min, x_max = X[:,0].min()-padding, X[:,0].max()+padding
    y_min, y_max = X[:,1].min()-padding, X[:,1].max()+padding
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                            np.arange(y_min, y_max, h))
    grid = np.c_[xx.ravel(), yy.ravel()]

    # -- Prédiction robuste (proba > decision_function > predict)
    Z = None
    if hasattr(pipe, "predict_proba"):
        proba = pipe.predict_proba(grid)
        # binaire : colonnes [p(classe0), p(classe1)] ou vecteur

```

```

        Z = proba[:,1] if proba.ndim == 2 and proba.shape[1] > 1 else proba.
        ravel()
        elif hasattr(pipe, "decision_function"):
            s = pipe.decision_function(grid).ravel()
            # Normalisation sigmoïde légère sans SciPy
            Z = 1 / (1 + np.exp(-s))
        else:
            # chute de secours : classes {0,1}
            Z = pipe.predict(grid).ravel().astype(float)

    Z_grid = Z.reshape(xx.shape)

    # -- Fond coloré + ligne de frontière
    # si Z est proba, on trace la courbe de niveau au seuil; sinon on sépare 0/1
    if np.unique(Z).size > 2 or (Z.min() >= 0 and Z.max() <= 1):
        ax.contourf(xx, yy, Z_grid, levels=50, alpha=0.25)
        ax.contour(xx, yy, (Z_grid >= threshold).astype(int), levels=[0.5],
        colors="k", linewidths=1)
    else:
        ax.contourf(xx, yy, Z_grid, levels=[-0.5,0.5,1.5], alpha=0.25)
        ax.contour(xx, yy, Z_grid, levels=[0.5], colors="k", linewidths=1)

    # -- Nuage de points (si y fourni)
    if y is not None:
        ax.scatter(X[:,0], X[:,1], c=y, s=25, edgecolor="k", linewidth=0.5)

    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    if title:
        ax.set_title(title)
    ax.set_aspect("equal", adjustable="box")
    plt.tight_layout()
    return ax

def plot_examples(X,y,nb_columns=25):
    plt.figure(figsize=(15,15))
    for i in range(nb_columns):
        plt.subplot(5,5,i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        # cv2 lit met les images en BGR et matplotlib lit du RGB
        img_rgb = cv2.cvtColor(X[i], cv2.COLOR_BGR2RGB)
        plt.imshow(img_rgb/255., cmap=plt.cm.binary)
        plt.xlabel('classe ' + str(y[i]))

```

```
[4]: zip_files=["Data_sheep_cat_elephant_with_caption_200.zip",
               "Data_sheep_cat_elephant_with_caption_400.zip",
               "Data_sheep_cat_elephant_with_caption_600.zip"]

for file in zip_files:
    !wget https://www.lirmm.fr/~poncelet/Ressources/{file}

    with zipfile.ZipFile(file, "r") as zip_ref:
        zip_ref.extractall("Data_Project")

    os.remove(file)
```

```
--2026-04-05 21:54:50-- https://www.lirmm.fr/~poncelet/Ressources/Data_sheep_cat_elephant_with_caption_200.zip
Résolution de www.lirmm.fr (www.lirmm.fr)... 193.49.104.251
Connexion à www.lirmm.fr (www.lirmm.fr)|193.49.104.251|:443... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 115482858 (110M) [application/zip]
Enregistre : 'Data_sheep_cat_elephant_with_caption_200.zip'
```

```
eep_cat_elephant_wi    9%[>                                ] 10,15M  2,55MB/s    tps 35s
^C
```

```
-----
BadZipFile                                Traceback (most recent call last)
Cell In[4], line 8
      5 for file in zip_files:
      6     get_ipython().system('wget https://www.lirmm.fr/~poncelet/Ressource /
↳ {file}')
----> 8     with zipfile.ZipFile(file, "r") as zip_ref:
      9         zip_ref.extractall("Data_Project")
     11     os.remove(file)

File /usr/lib/python3.10/zipfile.py:1288, in ZipFile.__init__(self, file, mode,
↳ compression, allowZip64, compresslevel, strict_timestamps)
     1286 try:
     1287     if mode == 'r':
-> 1288         self._RealGetContents()
     1289     elif mode in ('w', 'x'):
     1290         # set the modified flag so central directory gets written
     1291         # even if no files are added to the archive
     1292         self._didModify = True

File /usr/lib/python3.10/zipfile.py:1355, in ZipFile._RealGetContents(self)
     1353     raise BadZipFile("File is not a zip file")
     1354 if not endrec:
-> 1355     raise BadZipFile("File is not a zip file")
```

```

1356 if self.debug > 1:
1357     print(endrec)

```

BadZipFile: File is not a zip file

2 Resize images

```

[6]: IMG_SIZE=128
mypaths=['Data_Project/images/elephant', 'Data_Project/images/cat',
↪ 'Data_Project/images/sheep']

for path in mypaths:
    onlyfiles = [ f for f in listdir(path) if isfile(join(path,f)) ]
    images = np.empty(len(onlyfiles), dtype=object)
    for n in range(0, len(onlyfiles)):
        images[n] = cv2.imread( join(path,onlyfiles[n]) )
        images[n] = cv2.resize(images[n], (IMG_SIZE, IMG_SIZE))

```

```

[7]: #there is a slight modification from the code we copied
def create_training_data(path_data, list_classes):
    training_data=[]
    for classes in list_classes:
        path=os.path.join(path_data, classes)
        class_num=list_classes.index(classes)
        for img in os.listdir(path):
            try:
                img_array = cv2.imread(os.path.join(path,img), cv2.IMREAD_COLOR) #we
↪ can read colors now forcing 3 channels
                new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
                training_data.append([new_array, class_num])
            except Exception as e:
                pass
    return training_data

def create_X_y (path_data, list_classes):
    # récupération des données
    training_data=create_training_data(path_data, list_classes)
    # tri des données
    random.shuffle(training_data)
    # création de X et y
    X=[]
    y=[]
    for features, label in training_data:
        X.append(features)
        y.append(label)

```



```
X=np.array(X).reshape(-1,IMG_SIZE, IMG_SIZE, 3)
y=np.array(y)
return X,y
```

3 Loading the classes

```
[41]: my_path = "Data_Project/images/"
my_classes = ['elephant', 'cat', 'sheep']

X, y = create_X_y(my_path, my_classes)

plot_examples(X, y)

print("Nombre de données :", X.shape[0])
print("Taille d'une image :", X[0].shape)
print("Distribution des classes :")
print(pd.Series(y).value_counts())
```

Nombre de données : 1800

Taille d'une image : (128, 128, 3)

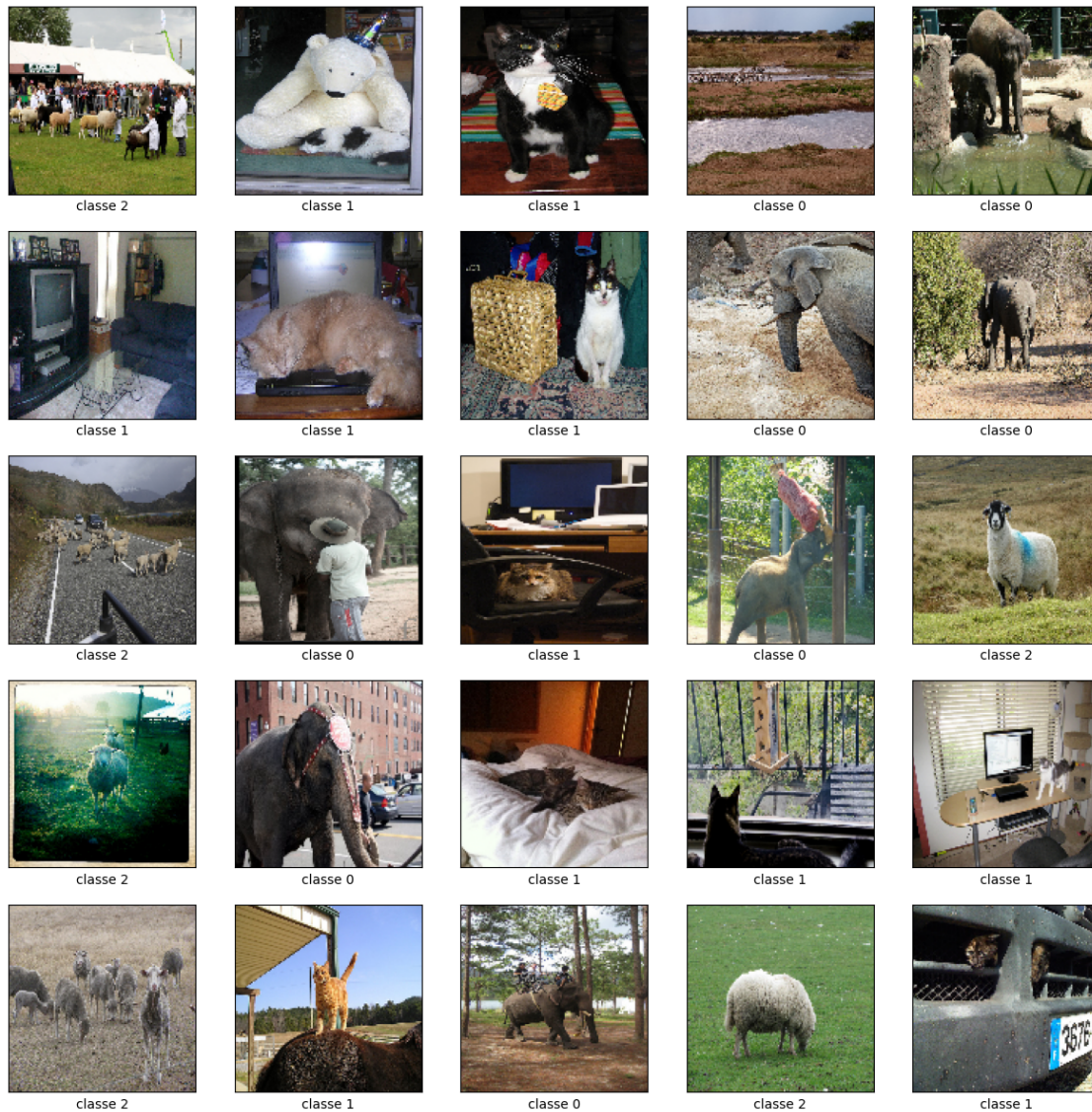
Distribution des classes :

2 600

1 600

0 600

Name: count, dtype: int64



4 Normalize data

```
[42]: X=X.astype('float')
      X=X/255.0
```

5 Building the CNN model

```
[8]: image_size=128

input = Input(shape=(image_size, image_size, 3), name="Input_layer")
```

```

x = Conv2D(32, (3, 3), activation="relu", name="Conv2D_1")(input)
x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_1")(x)
x = Flatten(name="Flatten")(x)
x = Dense(64, activation="relu", name="dense_1")(x)

output = Dense(3, activation="softmax", name="Output_Layer")(x)

model = Model(inputs=input, outputs=output, name="Baseline_CNN")

```

W0000 00:00:1775401944.691248 27213 gpu_device.cc:2344] Cannot dlopen some GPU libraries. Please make sure the missing libraries mentioned above are installed properly if you would like to use GPU. Follow the guide at <https://www.tensorflow.org/install/gpu> for how to download and setup the required libraries for your platform.
Skipping registering GPU devices...

6 Compiling the Model

```

[9]: model.compile(optimizer=Adam(1e-3),
                  loss="sparse_categorical_crossentropy", #categorical entropy
                  ↪doesn't work, forgot i needed the data to be encoded :(
                  metrics=["accuracy"])

```

```

[10]: model.summary()

```

Model: "Baseline_CNN"

Layer (type)	Output Shape	Param #
Input_layer (InputLayer)	(None, 128, 128, 3)	0
Conv2D_1 (Conv2D)	(None, 126, 126, 32)	896
MaxPool2D_1 (MaxPooling2D)	(None, 63, 63, 32)	0
Flatten (Flatten)	(None, 127008)	0
dense_1 (Dense)	(None, 64)	8,128,576
Output_Layer (Dense)	(None, 3)	195

Total params: 8,129,667 (31.01 MB)

Trainable params: 8,129,667 (31.01 MB)

Non-trainable params: 0 (0.00 B)

```
[11]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, stratify=y, random_state=42
    )

    # Entraînement
    epochs = 40
    batch_size = 32
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.2,
        verbose=1
    )

    # Courbes loss/accuracy
    plot_history_simple(history)

    # Évaluation sur X_test
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    print("Test accuracy:", round(test_acc, 3))
```

Epoch 1/40

36/36 3s 83ms/step -

accuracy: 0.4557 - loss: 2.3658 - val_accuracy: 0.6597 - val_loss: 0.8895

Epoch 2/40

36/36 3s 81ms/step -

accuracy: 0.6918 - loss: 0.7743 - val_accuracy: 0.6458 - val_loss: 0.8537

Epoch 3/40

36/36 3s 79ms/step -

accuracy: 0.8038 - loss: 0.5655 - val_accuracy: 0.6736 - val_loss: 0.8367

Epoch 4/40

36/36 3s 79ms/step -

accuracy: 0.8941 - loss: 0.3724 - val_accuracy: 0.6493 - val_loss: 0.8698

Epoch 5/40

36/36 3s 80ms/step -

accuracy: 0.9089 - loss: 0.3050 - val_accuracy: 0.6250 - val_loss: 0.8652

Epoch 6/40

36/36 3s 80ms/step -

accuracy: 0.9288 - loss: 0.2637 - val_accuracy: 0.5729 - val_loss: 1.1386

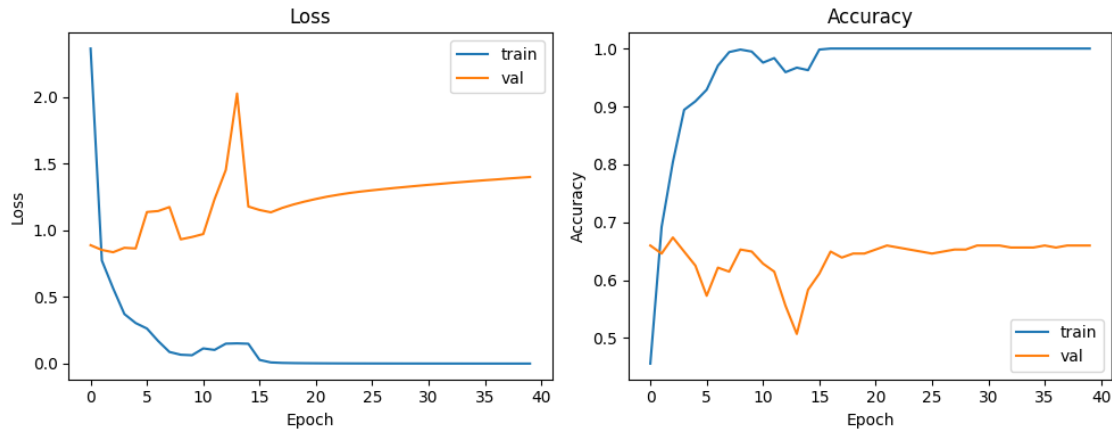
Epoch 7/40

36/36 3s 80ms/step -

accuracy: 0.9705 - loss: 0.1697 - val_accuracy: 0.6215 - val_loss: 1.1461
 Epoch 8/40
 36/36 3s 80ms/step -
 accuracy: 0.9939 - loss: 0.0879 - val_accuracy: 0.6146 - val_loss: 1.1758
 Epoch 9/40
 36/36 3s 80ms/step -
 accuracy: 0.9983 - loss: 0.0671 - val_accuracy: 0.6528 - val_loss: 0.9335
 Epoch 10/40
 36/36 3s 81ms/step -
 accuracy: 0.9948 - loss: 0.0633 - val_accuracy: 0.6493 - val_loss: 0.9510
 Epoch 11/40
 36/36 3s 80ms/step -
 accuracy: 0.9757 - loss: 0.1146 - val_accuracy: 0.6285 - val_loss: 0.9733
 Epoch 12/40
 36/36 3s 82ms/step -
 accuracy: 0.9835 - loss: 0.1033 - val_accuracy: 0.6146 - val_loss: 1.2347
 Epoch 13/40
 36/36 3s 82ms/step -
 accuracy: 0.9592 - loss: 0.1501 - val_accuracy: 0.5556 - val_loss: 1.4542
 Epoch 14/40
 36/36 3s 82ms/step -
 accuracy: 0.9670 - loss: 0.1524 - val_accuracy: 0.5069 - val_loss: 2.0280
 Epoch 15/40
 36/36 3s 81ms/step -
 accuracy: 0.9627 - loss: 0.1496 - val_accuracy: 0.5833 - val_loss: 1.1798
 Epoch 16/40
 36/36 3s 82ms/step -
 accuracy: 0.9983 - loss: 0.0281 - val_accuracy: 0.6111 - val_loss: 1.1538
 Epoch 17/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 0.0093 - val_accuracy: 0.6493 - val_loss: 1.1364
 Epoch 18/40
 36/36 3s 82ms/step -
 accuracy: 1.0000 - loss: 0.0057 - val_accuracy: 0.6389 - val_loss: 1.1690
 Epoch 19/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 0.6458 - val_loss: 1.1950
 Epoch 20/40
 36/36 3s 82ms/step -
 accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 0.6458 - val_loss: 1.2171
 Epoch 21/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 0.0029 - val_accuracy: 0.6528 - val_loss: 1.2364
 Epoch 22/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0025 - val_accuracy: 0.6597 - val_loss: 1.2541
 Epoch 23/40
 36/36 3s 82ms/step -

accuracy: 1.0000 - loss: 0.0021 - val_accuracy: 0.6562 - val_loss: 1.2685
 Epoch 24/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0019 - val_accuracy: 0.6528 - val_loss: 1.2816
 Epoch 25/40
 36/36 3s 82ms/step -
 accuracy: 1.0000 - loss: 0.0017 - val_accuracy: 0.6493 - val_loss: 1.2923
 Epoch 26/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0015 - val_accuracy: 0.6458 - val_loss: 1.3018
 Epoch 27/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 0.6493 - val_loss: 1.3107
 Epoch 28/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.6528 - val_loss: 1.3193
 Epoch 29/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 0.6528 - val_loss: 1.3272
 Epoch 30/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.6597 - val_loss: 1.3351
 Epoch 31/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 0.0010 - val_accuracy: 0.6597 - val_loss: 1.3428
 Epoch 32/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 9.3580e-04 - val_accuracy: 0.6597 - val_loss: 1.3496
 Epoch 33/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 8.7576e-04 - val_accuracy: 0.6562 - val_loss: 1.3572
 Epoch 34/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 8.2157e-04 - val_accuracy: 0.6562 - val_loss: 1.3639
 Epoch 35/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 7.7301e-04 - val_accuracy: 0.6562 - val_loss: 1.3706
 Epoch 36/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 7.2843e-04 - val_accuracy: 0.6597 - val_loss: 1.3771
 Epoch 37/40
 36/36 3s 81ms/step -
 accuracy: 1.0000 - loss: 6.8768e-04 - val_accuracy: 0.6562 - val_loss: 1.3830
 Epoch 38/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 6.5058e-04 - val_accuracy: 0.6597 - val_loss: 1.3897
 Epoch 39/40
 36/36 3s 81ms/step -

accuracy: 1.0000 - loss: 6.1643e-04 - val_accuracy: 0.6597 - val_loss: 1.3956
 Epoch 40/40
 36/36 3s 80ms/step -
 accuracy: 1.0000 - loss: 5.8512e-04 - val_accuracy: 0.6597 - val_loss: 1.4014



Test accuracy: 0.658

7 Conclusion

Le model montre des signes de overfitting, la courbe de la fonction de perte pour l'entraînement continue de baisser tandis que celle pour le jeu de test cesse de diminuer autour l'epoch 4. et pour la courbe de l'accuracy on observe pour celle de jeu de test qu'elle n'augmente après l'epoch 10.

7.1 Solution :

7.1.1 1) Réglage du modèle (ajout de dropout,...ect)

7.1.2 2) Augmentation de Données

7.1.3 3)Transfer learning

8 Réglage du modèle

8.0.1 1er essai

```
[12]: from tensorflow.keras import regularizers
      from tensorflow.keras.layers import Dropout
      from tensorflow.keras.regularizers import l2
      from tensorflow.keras.optimizers import SGD
      image_size=128

      input = Input(shape=(image_size, image_size, 3), name="Input_layer")
```

```

x = Conv2D(32, (3, 3), activation="relu", padding="same",
↳name="Conv2D_1")(input)
x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_1")(x)
x = Conv2D(64, (3, 3), activation="relu", padding="same", name="Conv2D_2")(x)
x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_2")(x)
x = Flatten(name="Flatten")(x)
x = Dense(64, activation="relu", kernel_regularizer=regularizers.l2(0.01),
↳name="dense_1")(x)
x = Dropout(0.5)(x)

output = Dense(3, activation="softmax", name="Output_Layer")(x)

model = Model(inputs=input, outputs=output, name="Deeper_CNN")

```

```

[13]: opt = SGD(learning_rate=0.01, momentum=0.9)
model.compile(optimizer=opt,
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

```

```

[14]: model.summary()

```

Model: "Deeper_CNN"

Layer (type)	Output Shape	Param #
Input_layer (InputLayer)	(None, 128, 128, 3)	0
Conv2D_1 (Conv2D)	(None, 128, 128, 32)	896
MaxPool2D_1 (MaxPooling2D)	(None, 64, 64, 32)	0
Conv2D_2 (Conv2D)	(None, 64, 64, 64)	18,496
MaxPool2D_2 (MaxPooling2D)	(None, 32, 32, 64)	0
Flatten (Flatten)	(None, 65536)	0
dense_1 (Dense)	(None, 64)	4,194,368
dropout (Dropout)	(None, 64)	0
Output_Layer (Dense)	(None, 3)	195

Total params: 4,213,955 (16.07 MB)

Trainable params: 4,213,955 (16.07 MB)

Non-trainable params: 0 (0.00 B)

```
[61]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, stratify=y, random_state=42
    )

    # Entraînement
    epochs = 40
    batch_size = 32
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.2,
        verbose=1
    )

    # Courbes loss/accuracy
    plot_history_simple(history)

    # Évaluation sur X_test
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    print("Test accuracy:", round(test_acc, 3))
```

Epoch 1/40

36/36 4s 111ms/step -

accuracy: 0.3490 - loss: 2.3799 - val_accuracy: 0.3715 - val_loss: 2.2686

Epoch 2/40

36/36 4s 109ms/step -

accuracy: 0.3767 - loss: 2.1969 - val_accuracy: 0.4132 - val_loss: 2.1029

Epoch 3/40

36/36 4s 108ms/step -

accuracy: 0.4062 - loss: 2.0356 - val_accuracy: 0.3576 - val_loss: 1.9684

Epoch 4/40

36/36 4s 112ms/step -

accuracy: 0.3785 - loss: 1.9075 - val_accuracy: 0.4549 - val_loss: 1.8356

Epoch 5/40

36/36 4s 111ms/step -

accuracy: 0.4262 - loss: 1.7784 - val_accuracy: 0.4201 - val_loss: 1.7119

Epoch 6/40

36/36 4s 110ms/step -

accuracy: 0.4366 - loss: 1.6762 - val_accuracy: 0.4896 - val_loss: 1.6175

Epoch 7/40

36/36 4s 109ms/step -
 accuracy: 0.4514 - loss: 1.5827 - val_accuracy: 0.5417 - val_loss: 1.4688
 Epoch 8/40
 36/36 4s 110ms/step -
 accuracy: 0.5026 - loss: 1.4613 - val_accuracy: 0.5938 - val_loss: 1.3529
 Epoch 9/40
 36/36 4s 113ms/step -
 accuracy: 0.5703 - loss: 1.3333 - val_accuracy: 0.5972 - val_loss: 1.2352
 Epoch 10/40
 36/36 4s 108ms/step -
 accuracy: 0.5859 - loss: 1.2688 - val_accuracy: 0.6181 - val_loss: 1.1911
 Epoch 11/40
 36/36 4s 107ms/step -
 accuracy: 0.6007 - loss: 1.2064 - val_accuracy: 0.6215 - val_loss: 1.1213
 Epoch 12/40
 36/36 4s 107ms/step -
 accuracy: 0.6189 - loss: 1.1451 - val_accuracy: 0.5521 - val_loss: 1.1781
 Epoch 13/40
 36/36 4s 107ms/step -
 accuracy: 0.6094 - loss: 1.1247 - val_accuracy: 0.6215 - val_loss: 1.0787
 Epoch 14/40
 36/36 4s 108ms/step -
 accuracy: 0.6328 - loss: 1.0927 - val_accuracy: 0.6701 - val_loss: 1.0243
 Epoch 15/40
 36/36 4s 108ms/step -
 accuracy: 0.6528 - loss: 1.0101 - val_accuracy: 0.6215 - val_loss: 1.0307
 Epoch 16/40
 36/36 4s 107ms/step -
 accuracy: 0.6875 - loss: 0.9501 - val_accuracy: 0.6424 - val_loss: 0.9605
 Epoch 17/40
 36/36 4s 107ms/step -
 accuracy: 0.6884 - loss: 0.8945 - val_accuracy: 0.6458 - val_loss: 0.9925
 Epoch 18/40
 36/36 4s 108ms/step -
 accuracy: 0.7526 - loss: 0.8001 - val_accuracy: 0.6458 - val_loss: 1.0078
 Epoch 19/40
 36/36 4s 109ms/step -
 accuracy: 0.7648 - loss: 0.7761 - val_accuracy: 0.5312 - val_loss: 1.2254
 Epoch 20/40
 36/36 4s 109ms/step -
 accuracy: 0.7908 - loss: 0.7281 - val_accuracy: 0.6840 - val_loss: 0.9807
 Epoch 21/40
 36/36 4s 108ms/step -
 accuracy: 0.8238 - loss: 0.6545 - val_accuracy: 0.6840 - val_loss: 0.9948
 Epoch 22/40
 36/36 4s 109ms/step -
 accuracy: 0.8507 - loss: 0.6129 - val_accuracy: 0.6424 - val_loss: 1.1195
 Epoch 23/40

36/36 4s 108ms/step -
accuracy: 0.8759 - loss: 0.5715 - val_accuracy: 0.6632 - val_loss: 1.0887
Epoch 24/40

36/36 4s 109ms/step -
accuracy: 0.8793 - loss: 0.5548 - val_accuracy: 0.5451 - val_loss: 1.4553
Epoch 25/40

36/36 4s 109ms/step -
accuracy: 0.8498 - loss: 0.6612 - val_accuracy: 0.6076 - val_loss: 1.2945
Epoch 26/40

36/36 4s 108ms/step -
accuracy: 0.8550 - loss: 0.6396 - val_accuracy: 0.6076 - val_loss: 1.6185
Epoch 27/40

36/36 4s 109ms/step -
accuracy: 0.8516 - loss: 0.6916 - val_accuracy: 0.6944 - val_loss: 1.1718
Epoch 28/40

36/36 4s 110ms/step -
accuracy: 0.8281 - loss: 0.7505 - val_accuracy: 0.6632 - val_loss: 1.2906
Epoch 29/40

36/36 4s 108ms/step -
accuracy: 0.8602 - loss: 0.6764 - val_accuracy: 0.6389 - val_loss: 1.3820
Epoch 30/40

36/36 4s 109ms/step -
accuracy: 0.9141 - loss: 0.5983 - val_accuracy: 0.6215 - val_loss: 1.4372
Epoch 31/40

36/36 4s 110ms/step -
accuracy: 0.8880 - loss: 0.6148 - val_accuracy: 0.6250 - val_loss: 1.4081
Epoch 32/40

36/36 4s 110ms/step -
accuracy: 0.9062 - loss: 0.6123 - val_accuracy: 0.6354 - val_loss: 1.3875
Epoch 33/40

36/36 4s 107ms/step -
accuracy: 0.9427 - loss: 0.5220 - val_accuracy: 0.6285 - val_loss: 1.5583
Epoch 34/40

36/36 4s 109ms/step -
accuracy: 0.9453 - loss: 0.5094 - val_accuracy: 0.6562 - val_loss: 1.3632
Epoch 35/40

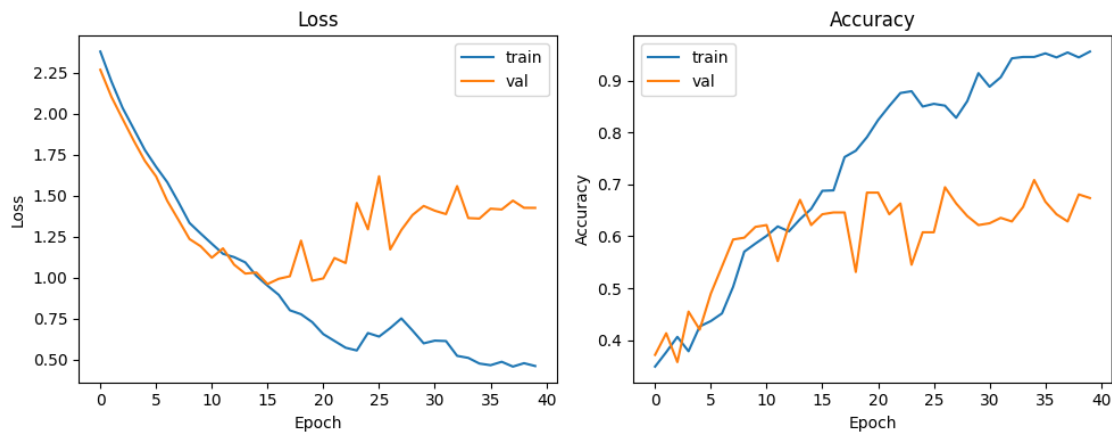
36/36 4s 109ms/step -
accuracy: 0.9453 - loss: 0.4749 - val_accuracy: 0.7083 - val_loss: 1.3602
Epoch 36/40

36/36 4s 108ms/step -
accuracy: 0.9523 - loss: 0.4649 - val_accuracy: 0.6667 - val_loss: 1.4205
Epoch 37/40

36/36 4s 107ms/step -
accuracy: 0.9444 - loss: 0.4858 - val_accuracy: 0.6424 - val_loss: 1.4158
Epoch 38/40

36/36 4s 108ms/step -
accuracy: 0.9540 - loss: 0.4563 - val_accuracy: 0.6285 - val_loss: 1.4697
Epoch 39/40

36/36 4s 108ms/step -
accuracy: 0.9444 - loss: 0.4771 - val_accuracy: 0.6806 - val_loss: 1.4259
Epoch 40/40
36/36 4s 106ms/step -
accuracy: 0.9557 - loss: 0.4598 - val_accuracy: 0.6736 - val_loss: 1.4254



Test accuracy: 0.703

9 Conclusion

le modèle prouve d'être mieux que le baseline mais il montre quelque signes de sur-apprentissage (la courbe d'accuracy, pour celle d'entraînement on remarque qu'elle converge vers la valeur 1 même si celle de validation plafonne environ 0.7, on observe des oscillation à partir de l'epoch 10 indiquant que la validation est trop petite). Ce modèle reste limité.

9.0.1 2ème essai

```
[67]: image_size=128

input = Input(shape=(image_size, image_size, 3), name="Input_layer")

x = Conv2D(32, (3, 3), activation="relu", padding="same",
    ↪name="Conv2D_1")(input)
x = BatchNormalization(name="BN_1")(x)
x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_1")(x)

#deepening my network

x = Conv2D(64, (3, 3), activation="relu", padding="same", name="Conv2D_2")(x)
x = BatchNormalization(name="BN_2")(x)
x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_2")(x)
```

```

x = GlobalAveragePooling2D(name="GAP")(x)
x = Dense(64, activation="relu", kernel_regularizer=regularizers.l2(0.01),
name="dense_1")(x)
x = Dropout(0.5)(x)

output = Dense(3, activation="softmax", name="Output_Layer")(x)

model = Model(inputs=input, outputs=output, name="Improved_CNN")

```

```

[68]: opt = SGD(learning_rate=0.01, momentum=0.9)
model.compile(optimizer=opt,
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

```

```

[69]: model.summary()

```

Model: "Improved_CNN"

Layer (type)	Output Shape	Param #
Input_layer (InputLayer)	(None, 128, 128, 3)	0
Conv2D_1 (Conv2D)	(None, 128, 128, 32)	896
BN_1 (BatchNormalization)	(None, 128, 128, 32)	128
MaxPool2D_1 (MaxPooling2D)	(None, 64, 64, 32)	0
Conv2D_2 (Conv2D)	(None, 64, 64, 64)	18,496
BN_2 (BatchNormalization)	(None, 64, 64, 64)	256
MaxPool2D_2 (MaxPooling2D)	(None, 32, 32, 64)	0
GAP (GlobalAveragePooling2D)	(None, 64)	0
dense_1 (Dense)	(None, 64)	4,160
dropout_4 (Dropout)	(None, 64)	0
Output_Layer (Dense)	(None, 3)	195

Total params: 24,131 (94.26 KB)

Trainable params: 23,939 (93.51 KB)

Non-trainable params: 192 (768.00 B)

```
[70]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, stratify=y, random_state=42
    )

    es = EarlyStopping(monitor="val_loss", patience=10,
                      restore_best_weights=True, verbose=1)

    rlrop = ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                             patience=5, min_lr=1e-6, verbose=1)

    # Entraînement
    epochs = 40
    batch_size = 32
    history = model.fit(
        X_train, y_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.2,
        callbacks=[es, rlrop],
        verbose=1
    )

    # Courbes loss/accuracy
    plot_history_simple(history)

    # Évaluation sur X_test
    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
    print("Test accuracy:", round(test_acc, 3))
```

Epoch 1/40

36/36 9s 222ms/step -

accuracy: 0.5069 - loss: 1.6768 - val_accuracy: 0.3681 - val_loss: 1.6828 -
learning_rate: 0.0100

Epoch 2/40

36/36 8s 210ms/step -

accuracy: 0.5642 - loss: 1.4685 - val_accuracy: 0.3785 - val_loss: 1.6026 -
learning_rate: 0.0100

Epoch 3/40

36/36 8s 214ms/step -

accuracy: 0.5738 - loss: 1.3557 - val_accuracy: 0.3819 - val_loss: 1.5237 -
learning_rate: 0.0100

Epoch 4/40

36/36 8s 218ms/step -
accuracy: 0.5877 - loss: 1.2921 - val_accuracy: 0.3542 - val_loss: 1.4875 -
learning_rate: 0.0100
Epoch 5/40

36/36 8s 222ms/step -
accuracy: 0.5998 - loss: 1.2195 - val_accuracy: 0.3646 - val_loss: 1.4243 -
learning_rate: 0.0100
Epoch 6/40

36/36 8s 214ms/step -
accuracy: 0.6181 - loss: 1.1730 - val_accuracy: 0.4167 - val_loss: 1.2868 -
learning_rate: 0.0100
Epoch 7/40

36/36 8s 218ms/step -
accuracy: 0.6207 - loss: 1.1146 - val_accuracy: 0.5556 - val_loss: 1.1475 -
learning_rate: 0.0100
Epoch 8/40

36/36 8s 225ms/step -
accuracy: 0.6050 - loss: 1.0873 - val_accuracy: 0.5694 - val_loss: 1.0825 -
learning_rate: 0.0100
Epoch 9/40

36/36 8s 226ms/step -
accuracy: 0.6181 - loss: 1.0513 - val_accuracy: 0.5868 - val_loss: 1.0677 -
learning_rate: 0.0100
Epoch 10/40

36/36 8s 222ms/step -
accuracy: 0.6372 - loss: 1.0152 - val_accuracy: 0.5938 - val_loss: 1.0112 -
learning_rate: 0.0100
Epoch 11/40

36/36 8s 216ms/step -
accuracy: 0.6458 - loss: 0.9741 - val_accuracy: 0.6493 - val_loss: 0.9395 -
learning_rate: 0.0100
Epoch 12/40

36/36 8s 219ms/step -
accuracy: 0.6337 - loss: 0.9666 - val_accuracy: 0.7014 - val_loss: 0.9067 -
learning_rate: 0.0100
Epoch 13/40

36/36 8s 218ms/step -
accuracy: 0.6267 - loss: 0.9347 - val_accuracy: 0.6840 - val_loss: 0.8915 -
learning_rate: 0.0100
Epoch 14/40

36/36 8s 214ms/step -
accuracy: 0.6536 - loss: 0.9138 - val_accuracy: 0.6562 - val_loss: 0.8624 -
learning_rate: 0.0100
Epoch 15/40

36/36 8s 212ms/step -
accuracy: 0.6389 - loss: 0.8917 - val_accuracy: 0.6042 - val_loss: 0.9260 -
learning_rate: 0.0100
Epoch 16/40

36/36 8s 213ms/step -
accuracy: 0.6424 - loss: 0.8859 - val_accuracy: 0.6562 - val_loss: 0.8683 -
learning_rate: 0.0100
Epoch 17/40

36/36 8s 213ms/step -
accuracy: 0.6606 - loss: 0.8640 - val_accuracy: 0.5903 - val_loss: 0.9326 -
learning_rate: 0.0100
Epoch 18/40

36/36 8s 213ms/step -
accuracy: 0.6615 - loss: 0.8547 - val_accuracy: 0.6354 - val_loss: 0.8622 -
learning_rate: 0.0100
Epoch 19/40

36/36 8s 212ms/step -
accuracy: 0.6589 - loss: 0.8495 - val_accuracy: 0.6701 - val_loss: 0.8608 -
learning_rate: 0.0100
Epoch 20/40

36/36 8s 213ms/step -
accuracy: 0.6589 - loss: 0.8414 - val_accuracy: 0.6632 - val_loss: 0.8354 -
learning_rate: 0.0100
Epoch 21/40

36/36 8s 214ms/step -
accuracy: 0.6641 - loss: 0.8247 - val_accuracy: 0.6562 - val_loss: 0.8248 -
learning_rate: 0.0100
Epoch 22/40

36/36 8s 214ms/step -
accuracy: 0.6675 - loss: 0.8249 - val_accuracy: 0.5660 - val_loss: 1.0558 -
learning_rate: 0.0100
Epoch 23/40

36/36 8s 214ms/step -
accuracy: 0.6736 - loss: 0.7964 - val_accuracy: 0.6875 - val_loss: 0.8142 -
learning_rate: 0.0100
Epoch 24/40

36/36 8s 215ms/step -
accuracy: 0.6797 - loss: 0.7988 - val_accuracy: 0.6632 - val_loss: 0.8291 -
learning_rate: 0.0100
Epoch 25/40

36/36 8s 216ms/step -
accuracy: 0.6832 - loss: 0.8017 - val_accuracy: 0.6910 - val_loss: 0.7655 -
learning_rate: 0.0100
Epoch 26/40

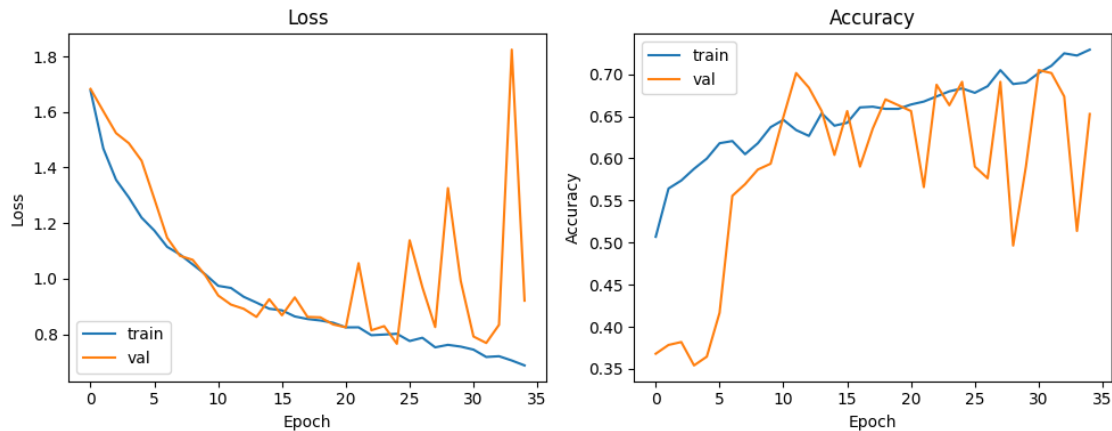
36/36 8s 214ms/step -
accuracy: 0.6780 - loss: 0.7756 - val_accuracy: 0.5903 - val_loss: 1.1381 -
learning_rate: 0.0100
Epoch 27/40

36/36 8s 216ms/step -
accuracy: 0.6858 - loss: 0.7873 - val_accuracy: 0.5764 - val_loss: 0.9688 -
learning_rate: 0.0100
Epoch 28/40


```

36/36          8s 216ms/step -
accuracy: 0.7049 - loss: 0.7529 - val_accuracy: 0.6910 - val_loss: 0.8257 -
learning_rate: 0.0100
Epoch 29/40
36/36          8s 215ms/step -
accuracy: 0.6884 - loss: 0.7618 - val_accuracy: 0.4965 - val_loss: 1.3258 -
learning_rate: 0.0100
Epoch 30/40
36/36          0s 205ms/step -
accuracy: 0.7024 - loss: 0.7475
Epoch 30: ReduceLROnPlateau reducing learning rate to 0.004999999888241291.
36/36          8s 215ms/step -
accuracy: 0.6901 - loss: 0.7554 - val_accuracy: 0.5903 - val_loss: 0.9919 -
learning_rate: 0.0100
Epoch 31/40
36/36          8s 215ms/step -
accuracy: 0.7014 - loss: 0.7446 - val_accuracy: 0.7049 - val_loss: 0.7918 -
learning_rate: 0.0050
Epoch 32/40
36/36          8s 216ms/step -
accuracy: 0.7101 - loss: 0.7181 - val_accuracy: 0.7014 - val_loss: 0.7685 -
learning_rate: 0.0050
Epoch 33/40
36/36          8s 216ms/step -
accuracy: 0.7248 - loss: 0.7208 - val_accuracy: 0.6736 - val_loss: 0.8338 -
learning_rate: 0.0050
Epoch 34/40
36/36          8s 217ms/step -
accuracy: 0.7222 - loss: 0.7055 - val_accuracy: 0.5139 - val_loss: 1.8242 -
learning_rate: 0.0050
Epoch 35/40
36/36          0s 207ms/step -
accuracy: 0.7364 - loss: 0.6859
Epoch 35: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.
36/36          8s 217ms/step -
accuracy: 0.7292 - loss: 0.6877 - val_accuracy: 0.6528 - val_loss: 0.9209 -
learning_rate: 0.0050
Epoch 35: early stopping
Restoring model weights from the end of the best epoch: 25.

```



Test accuracy: 0.694

[71]: #hmmmmmm

10 Conclusion

Le modèle ,même amélioré, reste limité par la taille du dataset.

10.0.1 Fonction build model

```
[10]: def build_model(name) :
    if name == "baseline":
        input = Input(shape=(image_size, image_size, 3), name="Input_layer")
        x = Conv2D(32, (3, 3), activation="relu", name="Conv2D_1")(input)
        x = MaxPooling2D((2, 2), name="MaxPool2D_1")(x)
        x = Flatten(name="Flatten")(x)
        x = Dense(64, activation="relu", name="dense_1")(x)
        out = Dense(3, activation="softmax", name="Output_Layer")(x)
        model = Model(inputs=input, outputs=out, name="Baseline_CNN")
        model.compile(optimizer=Adam(learning_rate=0.001),
                      loss="sparse_categorical_crossentropy",
                      metrics=["accuracy"])

    elif name == "deeper_CNN":
        input = Input(shape=(image_size, image_size, 3), name="Input_layer")
        x = Conv2D(32, (3, 3), activation="relu", padding="same",
        ↪name="Conv2D_1")(input)
        x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_1")(x)
        x = Conv2D(64, (3, 3), activation="relu", padding="same",
        ↪name="Conv2D_2")(x)
        x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_2")(x)
```

```

        x = Flatten(name="Flatten")(x)
        x = Dense(64, activation="relu", kernel_regularizer=regularizers.l2(0.
↳01), name="dense_1")(x)
        x = Dropout(0.3)(x)
        output = Dense(3, activation="softmax", name="Output_Layer")(x)
        model = Model(inputs=input, outputs=output, name="Deeper_CNN")
        opt = SGD(learning_rate=0.001, momentum=0.9)
        model.compile(optimizer=opt,
                        loss="sparse_categorical_crossentropy",
                        metrics=["accuracy"])

    elif name == "improved_CNN":
        input = Input(shape=(image_size, image_size, 3), name="Input_layer")
        x = Conv2D(32, (3, 3), activation="relu", padding="same",
↳name="Conv2D_1")(input)
        x = BatchNormalization(name="BN_1")(x)
        x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_1")(x)
        x = Conv2D(64, (3, 3), activation="relu", padding="same",
↳name="Conv2D_2")(x)
        x = BatchNormalization(name="BN_2")(x)
        x = MaxPooling2D(pool_size=(2, 2), name="MaxPool2D_2")(x)
        x = GlobalAveragePooling2D(name="GAP")(x)
        x = Dense(64, activation="relu", kernel_regularizer=regularizers.l2(0.
↳01), name="dense_1")(x)
        x = Dropout(0.3)(x)
        output = Dense(3, activation="softmax", name="Output_Layer")(x)
        model = Model(inputs=input, outputs=output, name="Improved_CNN")
        opt = SGD(learning_rate=0.001, momentum=0.9)
        model.compile(optimizer=opt,
                        loss="sparse_categorical_crossentropy",
                        metrics=["accuracy"])

    else :
        raise ValueError(f"Unknown model name '{name}'. Choose: baseline |
↳deeper_CNN | improved_CNN")

    return model

def train_cnn_model(model, train_generator, val_data, model_path,
                    epochs=10, patience=3):
    """
    Entraîne un modèle CNN avec un générateur de données et des callbacks.

    Args:
        model: Modèle CNN à entraîner.
        train_generator: Générateur d'images pour l'entraînement.
        val_data: Données de validation (X_test, y_test).
        model_path (str): Chemin de sauvegarde du modèle.

```

```

        epochs (int): Nombre d'époques d'entraînement.
        patience (int): Patience pour l'arrêt anticipé.

    Returns:
        Modèle entraîné avec les meilleurs poids restaurés.
    """
    callbacks = [
        EarlyStopping(monitor='val_loss', patience=patience,
                      restore_best_weights=True, verbose=1),
        ModelCheckpoint(model_path, monitor='val_loss',
                       save_best_only=True, verbose=1)
    ]

    history = model.fit(
        train_generator,
        epochs=epochs,
        validation_data=val_data,
        callbacks=callbacks,
        verbose=1
    )

    return model, history

```

11 Augmentation de Données via ImageDataGenerator

```

[11]: tf.keras.preprocessing.image.ImageDataGenerator(
featurewise_center=False, # Centrer les données globalement
samplewise_center=False, # Centrer chaque échantillon
featurewise_std_normalization=False, # Normaliser globalement
samplewise_std_normalization=False, # Normaliser chaque échantillon
zca_whitening=False, # Blanchiment ZCA
zca_epsilon=1e-06, # Epsilon pour ZCA
rotation_range=0, # Rotations aléatoires
width_shift_range=0.0, # Décalages horizontaux
height_shift_range=0.0, # Décalages verticaux
brightness_range=None, # Ajustement de luminosité
shear_range=0.0, # Cisaillement
zoom_range=0.0, # Zoom
channel_shift_range=0.0, # Décalage de canaux
fill_mode='nearest', # Remplissage
cval=0.0, # Valeur de remplissage
horizontal_flip=False, # Flip horizontal
vertical_flip=False, # Flip vertical
rescale=None, # Mise à l'échelle
preprocessing_function=None, # Fonction de prétraitement
data_format=None, # Format d'image

```

```
validation_split=0.0, # Fraction pour validation
dtype=None # Type de données
)
```

[11]: <keras.src.legacy.preprocessing.image.ImageDataGenerator at 0x799d75039390>

```
[8]: my_path = "Data_Project/images/"
my_classes = ['elephant', 'cat', 'sheep']

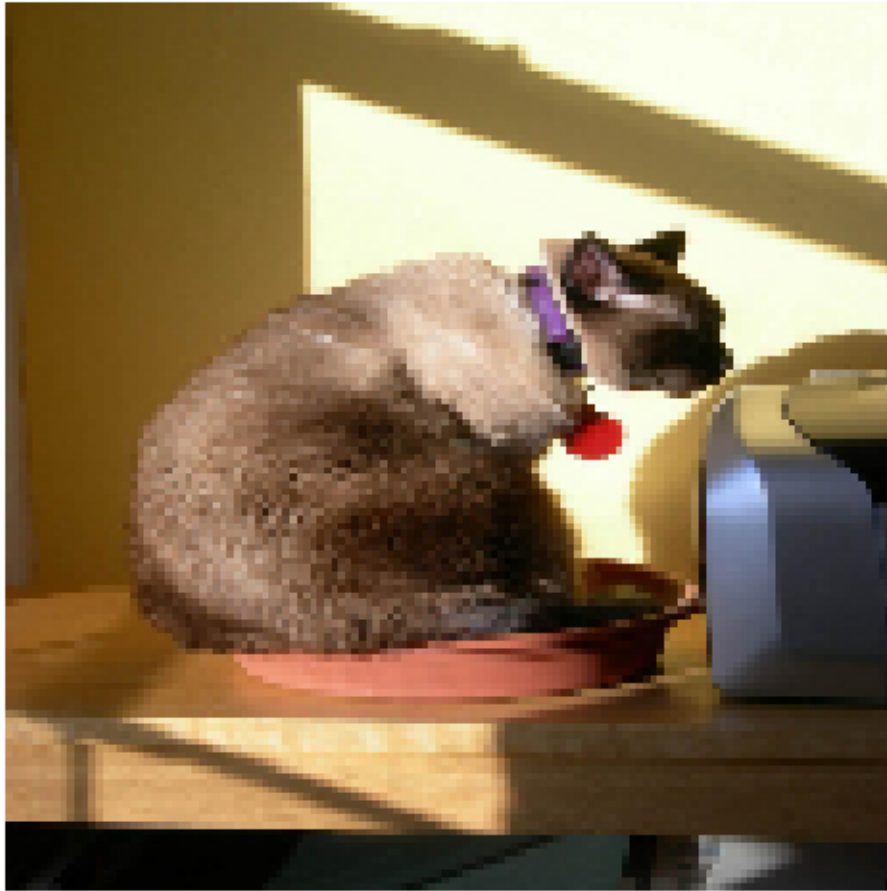
X, y = create_X_y(my_path, my_classes)
```

11.1 Découverte et Test de ImageDataGenerator

```
[94]: idx=10
img = X[idx]

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6,6))
plt.imshow(img_rgb)
plt.axis("off")
plt.show()
```



11.1.1 Flip horizontal et vertical

```
[95]: img_batch = np.expand_dims(img, axis=0)

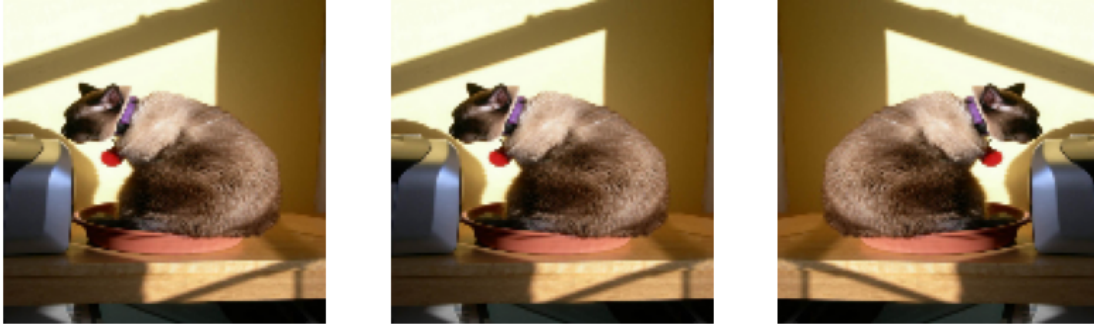
datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)
pic = datagen.flow(img_batch, batch_size=1)

plt.figure(figsize=(10,8))

for i in range(1,4):
    plt.subplot(1, 3, i)
    batch = next(pic) #next génère une image augmentée
    image_ = batch[0].astype('uint8')

    plt.imshow(cv2.cvtColor(image_, cv2.COLOR_BGR2RGB))
    plt.axis("off")

plt.show()
```



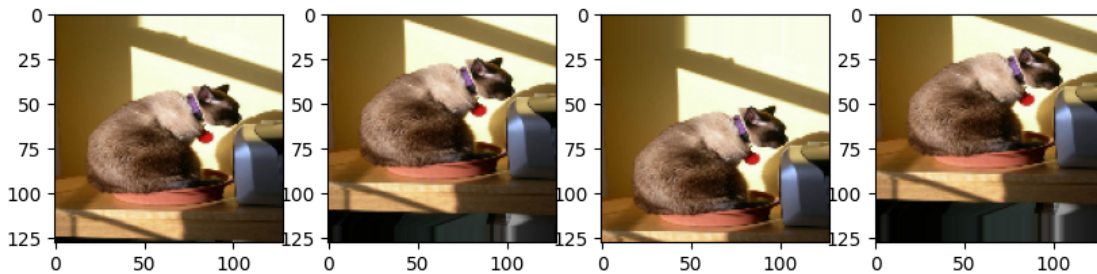
11.1.2 Décalage

```
[96]: datagen = ImageDataGenerator(width_shift_range= 20)
pic = datagen.flow(img_batch, batch_size=1)

plt.figure(figsize=(10,8))

for i in range(1,5):
    plt.subplot(1, 4, i)
    batch = next(pic)
    image_ = batch[0].astype('uint8')
    plt.imshow(cv2.cvtColor(image_, cv2.COLOR_BGR2RGB))

plt.show()
```



```
[97]: datagen = ImageDataGenerator(height_shift_range= 120)
pic = datagen.flow(img_batch, batch_size=1)

plt.figure(figsize=(10,8))

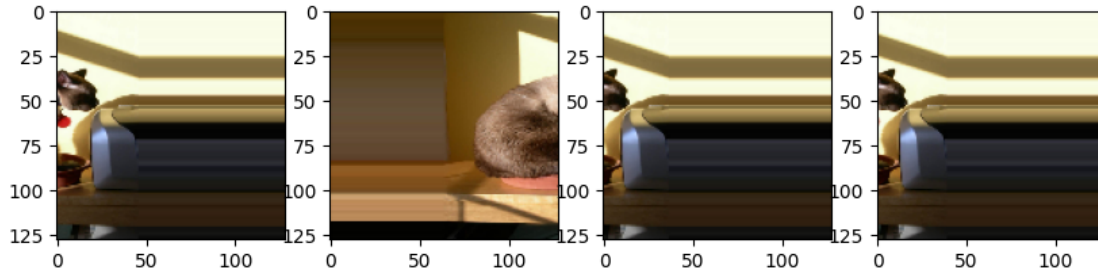
for i in range(1,5):
    plt.subplot(1, 4, i)
    batch = next(pic)
```

```

image_ = batch[0].astype('uint8')
plt.imshow(cv2.cvtColor(image_, cv2.COLOR_BGR2RGB))

plt.show()

```



```

[98]: def plot_augmented_images(X, y, augmentation_params={},
                                title="augmented images", batch_size=9):
    """
    Fonction pour afficher des images augmentées avec les paramètres spécifiés.

    Paramètres :
    - X (array) : tableau d'images (N,H,W,C)
    - y (array, optionnel) : labels correspondants.
    - augmentation_params (dict) : paramètres pour ImageDataGenerator.
    - title (str) : Titre du subplot pour la série d'images.
    - batch_size (int) : Taille du batch à afficher.
    """
    batch_size = min(batch_size, len(X), 9)
    X_sample = X[:batch_size]
    y_sample = y[:batch_size] if y is not None else None

    datagen = ImageDataGenerator(**augmentation_params)

    gen = datagen.flow(X_sample, y_sample, batch_size=batch_size, shuffle=False)
    X_batch, y_batch = next(gen)

    fig, ax = plt.subplots(3, 3, sharex=True, sharey=True, figsize=(6,6))
    fig.suptitle(title, fontsize=12)

    for i in range(batch_size):
        img_aug = X_batch[i].astype('uint8')

        if img_aug.shape[2] == 3:
            img_aug = cv2.cvtColor(img_aug, cv2.COLOR_BGR2RGB)

        row, col = divmod(i, 3)

```



```

ax[row, col].imshow(img_aug)
ax[row, col].axis('off')

# label si fourni
if y_sample is not None:
    ax[row, col].set_xlabel(str(y_sample[i]))

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

# Rotation
plot_augmented_images(X, y, {'rotation_range': 80}, title='Rotation')

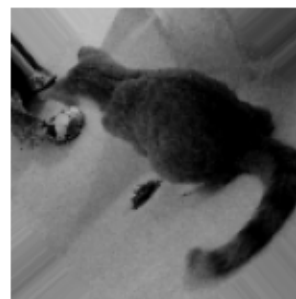
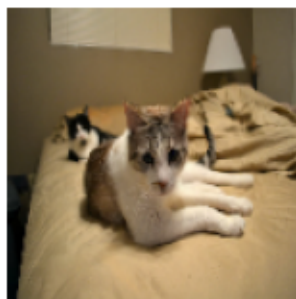
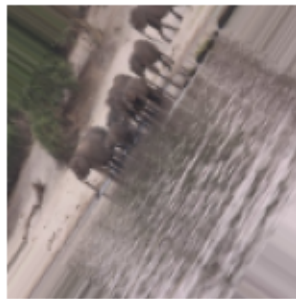
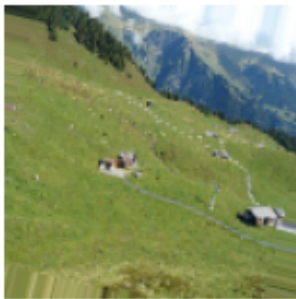
# Décalage
plot_augmented_images(X, y, {'width_shift_range': 20, 'height_shift_range': 20},
                        title='Décalage')

# Flip horizontal
plot_augmented_images(X, y, {'horizontal_flip': True}, title='Flip horizontal')

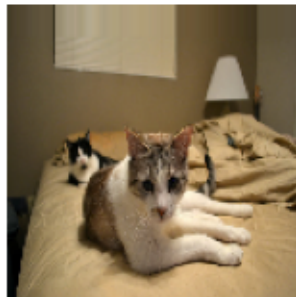
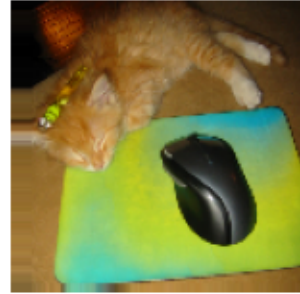
# Zoom
plot_augmented_images(X, y, {'zoom_range': 0.4}, title='Zoom')

```

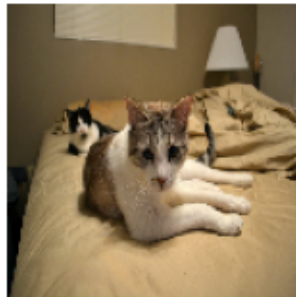
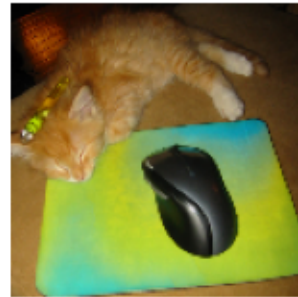
Rotation



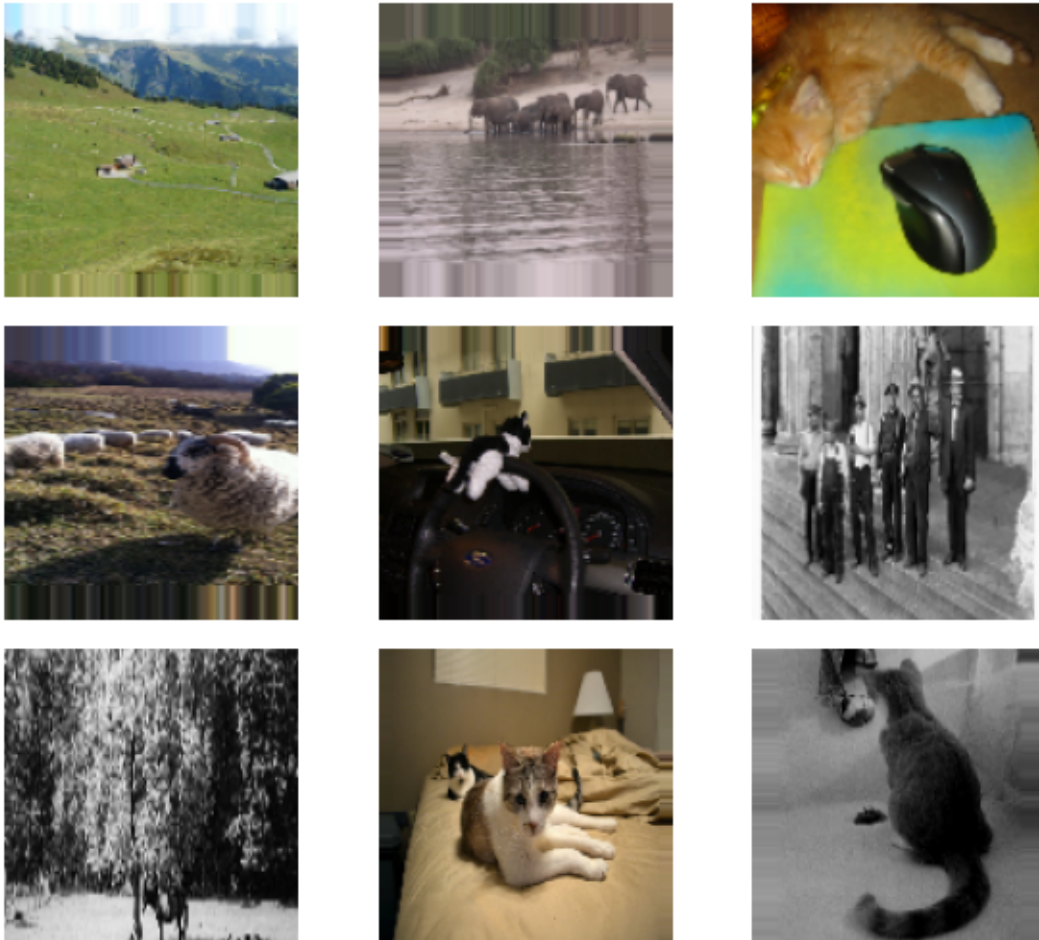
Décalage



Flip horizontal



Zoom



11.2 Essai avec ImageDataGenerator

```
[22]: image_size=128
X = X.astype("float32") / 255.0
X=X.astype('float')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

```
[17]: print("X_train range:", X_train.min(), X_train.max())
print("X_train dtype:", X_train.dtype)
print("X_train shape:", X_train.shape)
```

```
print("y_train unique:", np.unique(y_train))
print("y_train sample:", y_train[:5])
```

```
X_train range: 0.0 255.0
X_train dtype: float64
X_train shape: (1440, 128, 128, 3)
y_train unique: [0 1 2]
y_train sample: [2 2 1 0 2]
```

```
[28]: model_baseline = build_model("baseline")

datagen = ImageDataGenerator(
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True
)
train_generator = datagen.flow(X_train, y_train, batch_size=32, shuffle=True)

model_baseline, history_baseline = train_cnn_model(
    model_baseline,
    train_generator=train_generator,
    val_data=(X_test, y_test),
    model_path="best_baseline_aug.keras",
    epochs=40,
    patience=10
)
plot_history_simple(history_baseline)
_, acc_baseline = model_baseline.evaluate(X_test, y_test, verbose=0)
print("Baseline Test accuracy:", round(acc_baseline, 3))
```

```
Epoch 1/40
45/45          0s 95ms/step -
accuracy: 0.3790 - loss: 2.3811
Epoch 1: val_loss improved from None to 0.95958, saving model to
best_baseline_aug.keras
45/45          5s 111ms/step -
accuracy: 0.4486 - loss: 1.5521 - val_accuracy: 0.5278 - val_loss: 0.9596
Epoch 2/40
45/45          0s 95ms/step -
accuracy: 0.6180 - loss: 0.8957
Epoch 2: val_loss improved from 0.95958 to 0.83315, saving model to
best_baseline_aug.keras
45/45          5s 105ms/step -
accuracy: 0.6299 - loss: 0.8620 - val_accuracy: 0.6694 - val_loss: 0.8332
Epoch 3/40
45/45          0s 93ms/step -
accuracy: 0.6947 - loss: 0.7503
```

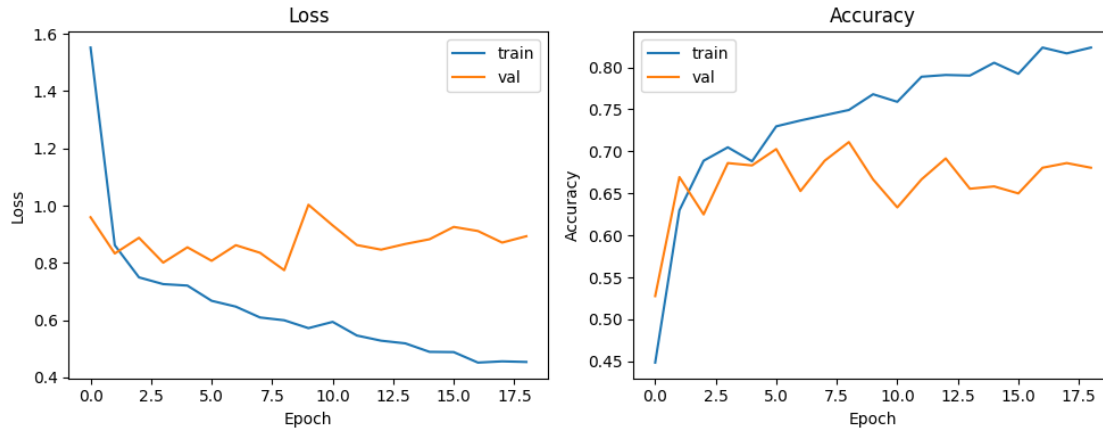

Epoch 3: val_loss did not improve from 0.83315
45/45 4s 98ms/step -
accuracy: 0.6889 - loss: 0.7497 - val_accuracy: 0.6250 - val_loss: 0.8879
Epoch 4/40
45/45 0s 94ms/step -
accuracy: 0.7081 - loss: 0.7215
Epoch 4: val_loss improved from 0.83315 to 0.80119, saving model to
best_baseline_aug.keras
45/45 5s 103ms/step -
accuracy: 0.7049 - loss: 0.7261 - val_accuracy: 0.6861 - val_loss: 0.8012
Epoch 5/40
45/45 0s 96ms/step -
accuracy: 0.6818 - loss: 0.7360
Epoch 5: val_loss did not improve from 0.80119
45/45 5s 101ms/step -
accuracy: 0.6882 - loss: 0.7210 - val_accuracy: 0.6833 - val_loss: 0.8547
Epoch 6/40
45/45 0s 97ms/step -
accuracy: 0.7122 - loss: 0.6840
Epoch 6: val_loss did not improve from 0.80119
45/45 5s 101ms/step -
accuracy: 0.7299 - loss: 0.6679 - val_accuracy: 0.7028 - val_loss: 0.8076
Epoch 7/40
45/45 0s 99ms/step -
accuracy: 0.7379 - loss: 0.6352
Epoch 7: val_loss did not improve from 0.80119
45/45 5s 103ms/step -
accuracy: 0.7368 - loss: 0.6475 - val_accuracy: 0.6528 - val_loss: 0.8620
Epoch 8/40
45/45 0s 94ms/step -
accuracy: 0.7429 - loss: 0.5983
Epoch 8: val_loss did not improve from 0.80119
45/45 4s 99ms/step -
accuracy: 0.7431 - loss: 0.6096 - val_accuracy: 0.6889 - val_loss: 0.8355
Epoch 9/40
45/45 0s 95ms/step -
accuracy: 0.7482 - loss: 0.5797
Epoch 9: val_loss improved from 0.80119 to 0.77474, saving model to
best_baseline_aug.keras
45/45 5s 105ms/step -
accuracy: 0.7493 - loss: 0.5999 - val_accuracy: 0.7111 - val_loss: 0.7747
Epoch 10/40
45/45 0s 97ms/step -
accuracy: 0.7797 - loss: 0.5617
Epoch 10: val_loss did not improve from 0.77474
45/45 5s 102ms/step -
accuracy: 0.7681 - loss: 0.5724 - val_accuracy: 0.6667 - val_loss: 1.0032
Epoch 11/40

```

45/45          0s 96ms/step -
accuracy: 0.7589 - loss: 0.5842
Epoch 11: val_loss did not improve from 0.77474
45/45          5s 100ms/step -
accuracy: 0.7590 - loss: 0.5943 - val_accuracy: 0.6333 - val_loss: 0.9312
Epoch 12/40
45/45          0s 95ms/step -
accuracy: 0.7737 - loss: 0.5771
Epoch 12: val_loss did not improve from 0.77474
45/45          4s 99ms/step -
accuracy: 0.7889 - loss: 0.5467 - val_accuracy: 0.6667 - val_loss: 0.8625
Epoch 13/40
45/45          0s 95ms/step -
accuracy: 0.7903 - loss: 0.5251
Epoch 13: val_loss did not improve from 0.77474
45/45          5s 99ms/step -
accuracy: 0.7910 - loss: 0.5286 - val_accuracy: 0.6917 - val_loss: 0.8465
Epoch 14/40
45/45          0s 97ms/step -
accuracy: 0.8071 - loss: 0.4901
Epoch 14: val_loss did not improve from 0.77474
45/45          5s 101ms/step -
accuracy: 0.7903 - loss: 0.5192 - val_accuracy: 0.6556 - val_loss: 0.8663
Epoch 15/40
45/45          0s 95ms/step -
accuracy: 0.7954 - loss: 0.4935
Epoch 15: val_loss did not improve from 0.77474
45/45          5s 100ms/step -
accuracy: 0.8056 - loss: 0.4896 - val_accuracy: 0.6583 - val_loss: 0.8828
Epoch 16/40
45/45          0s 95ms/step -
accuracy: 0.7963 - loss: 0.4737
Epoch 16: val_loss did not improve from 0.77474
45/45          5s 100ms/step -
accuracy: 0.7924 - loss: 0.4888 - val_accuracy: 0.6500 - val_loss: 0.9258
Epoch 17/40
45/45          0s 96ms/step -
accuracy: 0.8230 - loss: 0.4348
Epoch 17: val_loss did not improve from 0.77474
45/45          5s 100ms/step -
accuracy: 0.8236 - loss: 0.4521 - val_accuracy: 0.6806 - val_loss: 0.9116
Epoch 18/40
45/45          0s 97ms/step -
accuracy: 0.8303 - loss: 0.4513
Epoch 18: val_loss did not improve from 0.77474
45/45          5s 102ms/step -
accuracy: 0.8167 - loss: 0.4565 - val_accuracy: 0.6861 - val_loss: 0.8711
Epoch 19/40

```


45/45 0s 98ms/step -
accuracy: 0.8079 - loss: 0.4651
Epoch 19: val_loss did not improve from 0.77474
45/45 5s 102ms/step -
accuracy: 0.8236 - loss: 0.4544 - val_accuracy: 0.6806 - val_loss: 0.8934
Epoch 19: early stopping
Restoring model weights from the end of the best epoch: 9.



Baseline Test accuracy: 0.711

```
[29]: model_deeper = build_model("deeper_CNN")

datagen = ImageDataGenerator(
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True
)
train_generator = datagen.flow(X_train, y_train, batch_size=32, shuffle=True)

model_deeper, history_deeper = train_cnn_model(
    model_deeper,
    train_generator=train_generator,
    val_data=(X_test, y_test),
    model_path="best_deeper_aug.keras",
    epochs=40,
    patience=15
)
plot_history_simple(history_deeper)
_, acc_deeper = model_deeper.evaluate(X_test, y_test, verbose=0)
print("Deeper CNN Test accuracy:", round(acc_deeper, 3))
```

Epoch 1/40

```

45/45          0s 102ms/step -
accuracy: 0.3960 - loss: 2.3669
Epoch 1: val_loss improved from None to 2.30561, saving model to
best_deeper_aug.keras
45/45          6s 117ms/step -
accuracy: 0.4062 - loss: 2.3517 - val_accuracy: 0.4778 - val_loss: 2.3056
Epoch 2/40
45/45          0s 103ms/step -
accuracy: 0.4628 - loss: 2.2959
Epoch 2: val_loss improved from 2.30561 to 2.26886, saving model to
best_deeper_aug.keras
45/45          5s 113ms/step -
accuracy: 0.4569 - loss: 2.2848 - val_accuracy: 0.4417 - val_loss: 2.2689
Epoch 3/40
45/45          0s 104ms/step -
accuracy: 0.4786 - loss: 2.2533
Epoch 3: val_loss improved from 2.26886 to 2.22642, saving model to
best_deeper_aug.keras
45/45          5s 114ms/step -
accuracy: 0.4868 - loss: 2.2437 - val_accuracy: 0.5111 - val_loss: 2.2264
Epoch 4/40
45/45          0s 109ms/step -
accuracy: 0.5100 - loss: 2.2047
Epoch 4: val_loss improved from 2.22642 to 2.19313, saving model to
best_deeper_aug.keras
45/45          5s 119ms/step -
accuracy: 0.5104 - loss: 2.2020 - val_accuracy: 0.4889 - val_loss: 2.1931
Epoch 5/40
45/45          0s 109ms/step -
accuracy: 0.4945 - loss: 2.1864
Epoch 5: val_loss improved from 2.19313 to 2.13423, saving model to
best_deeper_aug.keras
45/45          5s 119ms/step -
accuracy: 0.5035 - loss: 2.1776 - val_accuracy: 0.5389 - val_loss: 2.1342
Epoch 6/40
45/45          0s 108ms/step -
accuracy: 0.5438 - loss: 2.1355
Epoch 6: val_loss improved from 2.13423 to 2.10820, saving model to
best_deeper_aug.keras
45/45          5s 118ms/step -
accuracy: 0.5382 - loss: 2.1249 - val_accuracy: 0.5528 - val_loss: 2.1082
Epoch 7/40
45/45          0s 109ms/step -
accuracy: 0.5672 - loss: 2.0834
Epoch 7: val_loss improved from 2.10820 to 2.06855, saving model to
best_deeper_aug.keras
45/45          5s 119ms/step -
accuracy: 0.5708 - loss: 2.0758 - val_accuracy: 0.5500 - val_loss: 2.0686

```

Epoch 8/40
45/45 0s 112ms/step -
accuracy: 0.5786 - loss: 2.0464
Epoch 8: val_loss improved from 2.06855 to 2.04766, saving model to
best_deeper_aug.keras
45/45 6s 121ms/step -
accuracy: 0.5667 - loss: 2.0551 - val_accuracy: 0.5611 - val_loss: 2.0477
Epoch 9/40
45/45 0s 112ms/step -
accuracy: 0.5760 - loss: 2.0200
Epoch 9: val_loss improved from 2.04766 to 1.96074, saving model to
best_deeper_aug.keras
45/45 6s 121ms/step -
accuracy: 0.5840 - loss: 2.0008 - val_accuracy: 0.6250 - val_loss: 1.9607
Epoch 10/40
45/45 0s 110ms/step -
accuracy: 0.6219 - loss: 1.9555
Epoch 10: val_loss did not improve from 1.96074
45/45 5s 118ms/step -
accuracy: 0.5847 - loss: 1.9794 - val_accuracy: 0.5861 - val_loss: 1.9737
Epoch 11/40
45/45 0s 109ms/step -
accuracy: 0.6198 - loss: 1.9205
Epoch 11: val_loss improved from 1.96074 to 1.87431, saving model to
best_deeper_aug.keras
45/45 5s 119ms/step -
accuracy: 0.6271 - loss: 1.9097 - val_accuracy: 0.6444 - val_loss: 1.8743
Epoch 12/40
45/45 0s 109ms/step -
accuracy: 0.6354 - loss: 1.8616
Epoch 12: val_loss did not improve from 1.87431
45/45 5s 117ms/step -
accuracy: 0.6361 - loss: 1.8770 - val_accuracy: 0.6222 - val_loss: 1.8869
Epoch 13/40
45/45 0s 108ms/step -
accuracy: 0.6705 - loss: 1.8500
Epoch 13: val_loss improved from 1.87431 to 1.81515, saving model to
best_deeper_aug.keras
45/45 5s 118ms/step -
accuracy: 0.6507 - loss: 1.8545 - val_accuracy: 0.6806 - val_loss: 1.8152
Epoch 14/40
45/45 0s 109ms/step -
accuracy: 0.6256 - loss: 1.8839
Epoch 14: val_loss did not improve from 1.81515
45/45 5s 117ms/step -
accuracy: 0.6472 - loss: 1.8390 - val_accuracy: 0.6389 - val_loss: 1.8499
Epoch 15/40
45/45 0s 110ms/step -

```

accuracy: 0.6397 - loss: 1.7959
Epoch 15: val_loss improved from 1.81515 to 1.77689, saving model to
best_deeper_aug.keras
45/45          5s 121ms/step -
accuracy: 0.6528 - loss: 1.7980 - val_accuracy: 0.6417 - val_loss: 1.7769
Epoch 16/40
45/45          0s 108ms/step -
accuracy: 0.6929 - loss: 1.7373
Epoch 16: val_loss did not improve from 1.77689
45/45          5s 115ms/step -
accuracy: 0.6840 - loss: 1.7528 - val_accuracy: 0.6278 - val_loss: 1.8292
Epoch 17/40
45/45          0s 108ms/step -
accuracy: 0.6474 - loss: 1.7574
Epoch 17: val_loss improved from 1.77689 to 1.71499, saving model to
best_deeper_aug.keras
45/45          5s 117ms/step -
accuracy: 0.6694 - loss: 1.7404 - val_accuracy: 0.6722 - val_loss: 1.7150
Epoch 18/40
45/45          0s 111ms/step -
accuracy: 0.6497 - loss: 1.7393
Epoch 18: val_loss did not improve from 1.71499
45/45          5s 118ms/step -
accuracy: 0.6438 - loss: 1.7525 - val_accuracy: 0.6306 - val_loss: 1.7753
Epoch 19/40
45/45          0s 109ms/step -
accuracy: 0.6821 - loss: 1.7264
Epoch 19: val_loss improved from 1.71499 to 1.67374, saving model to
best_deeper_aug.keras
45/45          5s 120ms/step -
accuracy: 0.6660 - loss: 1.7188 - val_accuracy: 0.6917 - val_loss: 1.6737
Epoch 20/40
45/45          0s 108ms/step -
accuracy: 0.6721 - loss: 1.6755
Epoch 20: val_loss did not improve from 1.67374
45/45          5s 116ms/step -
accuracy: 0.6840 - loss: 1.6662 - val_accuracy: 0.6917 - val_loss: 1.6743
Epoch 21/40
45/45          0s 109ms/step -
accuracy: 0.7079 - loss: 1.6386
Epoch 21: val_loss improved from 1.67374 to 1.63526, saving model to
best_deeper_aug.keras
45/45          5s 119ms/step -
accuracy: 0.6889 - loss: 1.6521 - val_accuracy: 0.7083 - val_loss: 1.6353
Epoch 22/40
45/45          0s 110ms/step -
accuracy: 0.6930 - loss: 1.6228
Epoch 22: val_loss improved from 1.63526 to 1.62458, saving model to

```

```

best_deeper_aug.keras
45/45          5s 120ms/step -
accuracy: 0.6917 - loss: 1.6224 - val_accuracy: 0.6806 - val_loss: 1.6246
Epoch 23/40
45/45          0s 110ms/step -
accuracy: 0.6863 - loss: 1.6317
Epoch 23: val_loss improved from 1.62458 to 1.60403, saving model to
best_deeper_aug.keras
45/45          5s 121ms/step -
accuracy: 0.6826 - loss: 1.6220 - val_accuracy: 0.7111 - val_loss: 1.6040
Epoch 24/40
45/45          0s 110ms/step -
accuracy: 0.6741 - loss: 1.6080
Epoch 24: val_loss improved from 1.60403 to 1.59484, saving model to
best_deeper_aug.keras
45/45          5s 119ms/step -
accuracy: 0.6917 - loss: 1.5812 - val_accuracy: 0.6833 - val_loss: 1.5948
Epoch 25/40
45/45          0s 111ms/step -
accuracy: 0.6968 - loss: 1.5617
Epoch 25: val_loss did not improve from 1.59484
45/45          5s 119ms/step -
accuracy: 0.7007 - loss: 1.5454 - val_accuracy: 0.6333 - val_loss: 1.6798
Epoch 26/40
45/45          0s 112ms/step -
accuracy: 0.6747 - loss: 1.5863
Epoch 26: val_loss improved from 1.59484 to 1.56099, saving model to
best_deeper_aug.keras
45/45          6s 122ms/step -
accuracy: 0.6875 - loss: 1.5534 - val_accuracy: 0.6778 - val_loss: 1.5610
Epoch 27/40
45/45          0s 113ms/step -
accuracy: 0.7035 - loss: 1.5161
Epoch 27: val_loss did not improve from 1.56099
45/45          5s 121ms/step -
accuracy: 0.6938 - loss: 1.5277 - val_accuracy: 0.6639 - val_loss: 1.5901
Epoch 28/40
45/45          0s 113ms/step -
accuracy: 0.6951 - loss: 1.5018
Epoch 28: val_loss improved from 1.56099 to 1.51862, saving model to
best_deeper_aug.keras
45/45          6s 124ms/step -
accuracy: 0.7090 - loss: 1.4917 - val_accuracy: 0.6972 - val_loss: 1.5186
Epoch 29/40
45/45          0s 111ms/step -
accuracy: 0.7137 - loss: 1.4824
Epoch 29: val_loss improved from 1.51862 to 1.50544, saving model to
best_deeper_aug.keras

```

```

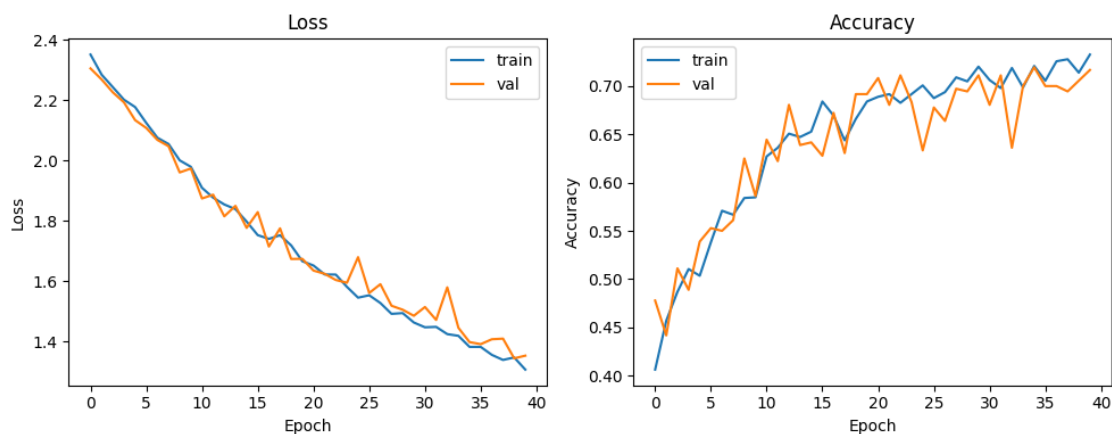
45/45          6s 122ms/step -
accuracy: 0.7049 - loss: 1.4943 - val_accuracy: 0.6944 - val_loss: 1.5054
Epoch 30/40
45/45          0s 114ms/step -
accuracy: 0.7243 - loss: 1.4664
Epoch 30: val_loss improved from 1.50544 to 1.48532, saving model to
best_deeper_aug.keras
45/45          6s 124ms/step -
accuracy: 0.7201 - loss: 1.4631 - val_accuracy: 0.7111 - val_loss: 1.4853
Epoch 31/40
45/45          0s 113ms/step -
accuracy: 0.7013 - loss: 1.4716
Epoch 31: val_loss did not improve from 1.48532
45/45          5s 121ms/step -
accuracy: 0.7063 - loss: 1.4472 - val_accuracy: 0.6806 - val_loss: 1.5145
Epoch 32/40
45/45          0s 113ms/step -
accuracy: 0.7051 - loss: 1.4353
Epoch 32: val_loss improved from 1.48532 to 1.47165, saving model to
best_deeper_aug.keras
45/45          6s 123ms/step -
accuracy: 0.6979 - loss: 1.4485 - val_accuracy: 0.7111 - val_loss: 1.4717
Epoch 33/40
45/45          0s 114ms/step -
accuracy: 0.7496 - loss: 1.4007
Epoch 33: val_loss did not improve from 1.47165
45/45          6s 121ms/step -
accuracy: 0.7188 - loss: 1.4246 - val_accuracy: 0.6361 - val_loss: 1.5797
Epoch 34/40
45/45          0s 114ms/step -
accuracy: 0.6989 - loss: 1.4116
Epoch 34: val_loss improved from 1.47165 to 1.44537, saving model to
best_deeper_aug.keras
45/45          6s 124ms/step -
accuracy: 0.6986 - loss: 1.4189 - val_accuracy: 0.7000 - val_loss: 1.4454
Epoch 35/40
45/45          0s 114ms/step -
accuracy: 0.7368 - loss: 1.3643
Epoch 35: val_loss improved from 1.44537 to 1.39800, saving model to
best_deeper_aug.keras
45/45          6s 125ms/step -
accuracy: 0.7208 - loss: 1.3822 - val_accuracy: 0.7194 - val_loss: 1.3980
Epoch 36/40
45/45          0s 115ms/step -
accuracy: 0.7022 - loss: 1.3991
Epoch 36: val_loss improved from 1.39800 to 1.39123, saving model to
best_deeper_aug.keras
45/45          6s 125ms/step -

```

```

accuracy: 0.7056 - loss: 1.3821 - val_accuracy: 0.7000 - val_loss: 1.3912
Epoch 37/40
45/45          0s 117ms/step -
accuracy: 0.7194 - loss: 1.3807
Epoch 37: val_loss did not improve from 1.39123
45/45          6s 125ms/step -
accuracy: 0.7257 - loss: 1.3557 - val_accuracy: 0.7000 - val_loss: 1.4075
Epoch 38/40
45/45          0s 115ms/step -
accuracy: 0.7244 - loss: 1.3450
Epoch 38: val_loss did not improve from 1.39123
45/45          6s 123ms/step -
accuracy: 0.7278 - loss: 1.3389 - val_accuracy: 0.6944 - val_loss: 1.4096
Epoch 39/40
45/45          0s 116ms/step -
accuracy: 0.7191 - loss: 1.3323
Epoch 39: val_loss improved from 1.39123 to 1.34424, saving model to
best_deeper_aug.keras
45/45          6s 126ms/step -
accuracy: 0.7139 - loss: 1.3472 - val_accuracy: 0.7056 - val_loss: 1.3442
Epoch 40/40
45/45          0s 115ms/step -
accuracy: 0.7279 - loss: 1.3205
Epoch 40: val_loss did not improve from 1.34424
45/45          6s 123ms/step -
accuracy: 0.7326 - loss: 1.3069 - val_accuracy: 0.7167 - val_loss: 1.3531
Restoring model weights from the end of the best epoch: 39.

```



Deeper CNN Test accuracy: 0.706

```
[30]: model_improved = build_model("improved_CNN")
```

```

datagen = ImageDataGenerator(
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True
)
train_generator = datagen.flow(X_train, y_train, batch_size=32, shuffle=True)

model_improved, history_improved = train_cnn_model(
    model_improved,
    train_generator=train_generator,
    val_data=(X_test, y_test),
    model_path="best_improved_aug.keras",
    epochs=40,
    patience=15
)
plot_history_simple(history_improved)
_, acc_improved = model_improved.evaluate(X_test, y_test, verbose=0)
print("Improved CNN Test accuracy:", round(acc_improved, 3))

```

Epoch 1/40

45/45 0s 195ms/step -

accuracy: 0.3927 - loss: 1.7605

Epoch 1: val_loss improved from None to 1.72330, saving model to
best_improved_aug.keras

45/45 10s 208ms/step -

accuracy: 0.4660 - loss: 1.6873 - val_accuracy: 0.3806 - val_loss: 1.7233

Epoch 2/40

45/45 0s 194ms/step -

accuracy: 0.5538 - loss: 1.5815

Epoch 2: val_loss improved from 1.72330 to 1.70542, saving model to
best_improved_aug.keras

45/45 9s 205ms/step -

accuracy: 0.5639 - loss: 1.5616 - val_accuracy: 0.4583 - val_loss: 1.7054

Epoch 3/40

45/45 0s 195ms/step -

accuracy: 0.5709 - loss: 1.5228

Epoch 3: val_loss improved from 1.70542 to 1.69994, saving model to
best_improved_aug.keras

45/45 9s 205ms/step -

accuracy: 0.5785 - loss: 1.5232 - val_accuracy: 0.3389 - val_loss: 1.6999

Epoch 4/40

45/45 0s 195ms/step -

accuracy: 0.6131 - loss: 1.4713

Epoch 4: val_loss improved from 1.69994 to 1.67910, saving model to
best_improved_aug.keras

45/45 9s 205ms/step -


```

accuracy: 0.5986 - loss: 1.4872 - val_accuracy: 0.3750 - val_loss: 1.6791
Epoch 5/40
45/45          0s 199ms/step -
accuracy: 0.5719 - loss: 1.4711
Epoch 5: val_loss improved from 1.67910 to 1.66492, saving model to
best_improved_aug.keras
45/45          9s 209ms/step -
accuracy: 0.5764 - loss: 1.4671 - val_accuracy: 0.3444 - val_loss: 1.6649
Epoch 6/40
45/45          0s 195ms/step -
accuracy: 0.5958 - loss: 1.4582
Epoch 6: val_loss improved from 1.66492 to 1.64187, saving model to
best_improved_aug.keras
45/45          9s 205ms/step -
accuracy: 0.6007 - loss: 1.4520 - val_accuracy: 0.4278 - val_loss: 1.6419
Epoch 7/40
45/45          0s 196ms/step -
accuracy: 0.5791 - loss: 1.4534
Epoch 7: val_loss improved from 1.64187 to 1.60083, saving model to
best_improved_aug.keras
45/45          9s 206ms/step -
accuracy: 0.5875 - loss: 1.4423 - val_accuracy: 0.4944 - val_loss: 1.6008
Epoch 8/40
45/45          0s 199ms/step -
accuracy: 0.6097 - loss: 1.4138
Epoch 8: val_loss improved from 1.60083 to 1.55648, saving model to
best_improved_aug.keras
45/45         10s 210ms/step -
accuracy: 0.6062 - loss: 1.4044 - val_accuracy: 0.5833 - val_loss: 1.5565
Epoch 9/40
45/45          0s 200ms/step -
accuracy: 0.6197 - loss: 1.4293
Epoch 9: val_loss improved from 1.55648 to 1.50007, saving model to
best_improved_aug.keras
45/45         10s 211ms/step -
accuracy: 0.6326 - loss: 1.3921 - val_accuracy: 0.6500 - val_loss: 1.5001
Epoch 10/40
45/45          0s 196ms/step -
accuracy: 0.5983 - loss: 1.3790
Epoch 10: val_loss did not improve from 1.50007
45/45          9s 205ms/step -
accuracy: 0.5986 - loss: 1.3866 - val_accuracy: 0.5944 - val_loss: 1.5002
Epoch 11/40
45/45          0s 196ms/step -
accuracy: 0.5858 - loss: 1.3903
Epoch 11: val_loss improved from 1.50007 to 1.38387, saving model to
best_improved_aug.keras
45/45          9s 206ms/step -

```

```

accuracy: 0.6194 - loss: 1.3611 - val_accuracy: 0.6472 - val_loss: 1.3839
Epoch 12/40
45/45          0s 194ms/step -
accuracy: 0.6596 - loss: 1.3404
Epoch 12: val_loss did not improve from 1.38387
45/45          9s 204ms/step -
accuracy: 0.6347 - loss: 1.3530 - val_accuracy: 0.6139 - val_loss: 1.4066
Epoch 13/40
45/45          0s 196ms/step -
accuracy: 0.6520 - loss: 1.3171
Epoch 13: val_loss did not improve from 1.38387
45/45          9s 206ms/step -
accuracy: 0.6424 - loss: 1.3254 - val_accuracy: 0.4917 - val_loss: 1.4815
Epoch 14/40
45/45          0s 194ms/step -
accuracy: 0.6543 - loss: 1.3073
Epoch 14: val_loss improved from 1.38387 to 1.34882, saving model to
best_improved_aug.keras
45/45          9s 205ms/step -
accuracy: 0.6458 - loss: 1.3221 - val_accuracy: 0.5889 - val_loss: 1.3488
Epoch 15/40
45/45          0s 196ms/step -
accuracy: 0.6526 - loss: 1.2918
Epoch 15: val_loss did not improve from 1.34882
45/45          9s 205ms/step -
accuracy: 0.6361 - loss: 1.2992 - val_accuracy: 0.4306 - val_loss: 1.6098
Epoch 16/40
45/45          0s 195ms/step -
accuracy: 0.6545 - loss: 1.2787
Epoch 16: val_loss improved from 1.34882 to 1.29398, saving model to
best_improved_aug.keras
45/45          9s 206ms/step -
accuracy: 0.6431 - loss: 1.2902 - val_accuracy: 0.6556 - val_loss: 1.2940
Epoch 17/40
45/45          0s 196ms/step -
accuracy: 0.6796 - loss: 1.2423
Epoch 17: val_loss did not improve from 1.29398
45/45          9s 206ms/step -
accuracy: 0.6507 - loss: 1.2769 - val_accuracy: 0.4972 - val_loss: 1.4926
Epoch 18/40
45/45          0s 195ms/step -
accuracy: 0.6520 - loss: 1.2601
Epoch 18: val_loss did not improve from 1.29398
45/45          9s 204ms/step -
accuracy: 0.6542 - loss: 1.2517 - val_accuracy: 0.5111 - val_loss: 1.4089
Epoch 19/40
45/45          0s 196ms/step -
accuracy: 0.6612 - loss: 1.2410

```

Epoch 19: val_loss did not improve from 1.29398
45/45 9s 206ms/step -
accuracy: 0.6514 - loss: 1.2537 - val_accuracy: 0.5778 - val_loss: 1.4574
Epoch 20/40
45/45 0s 195ms/step -
accuracy: 0.6830 - loss: 1.1945
Epoch 20: val_loss did not improve from 1.29398
45/45 9s 205ms/step -
accuracy: 0.6583 - loss: 1.2333 - val_accuracy: 0.5833 - val_loss: 1.3456
Epoch 21/40
45/45 0s 195ms/step -
accuracy: 0.6779 - loss: 1.2080
Epoch 21: val_loss did not improve from 1.29398
45/45 9s 205ms/step -
accuracy: 0.6556 - loss: 1.2329 - val_accuracy: 0.4667 - val_loss: 1.7110
Epoch 22/40
45/45 0s 195ms/step -
accuracy: 0.6704 - loss: 1.2194
Epoch 22: val_loss did not improve from 1.29398
45/45 9s 205ms/step -
accuracy: 0.6590 - loss: 1.2192 - val_accuracy: 0.5972 - val_loss: 1.2995
Epoch 23/40
45/45 0s 196ms/step -
accuracy: 0.6562 - loss: 1.2168
Epoch 23: val_loss improved from 1.29398 to 1.24474, saving model to
best_improved_aug.keras
45/45 9s 207ms/step -
accuracy: 0.6583 - loss: 1.2110 - val_accuracy: 0.6639 - val_loss: 1.2447
Epoch 24/40
45/45 0s 201ms/step -
accuracy: 0.6611 - loss: 1.2030
Epoch 24: val_loss did not improve from 1.24474
45/45 10s 211ms/step -
accuracy: 0.6569 - loss: 1.1969 - val_accuracy: 0.5778 - val_loss: 1.3196
Epoch 25/40
45/45 0s 196ms/step -
accuracy: 0.6540 - loss: 1.1827
Epoch 25: val_loss did not improve from 1.24474
45/45 9s 206ms/step -
accuracy: 0.6660 - loss: 1.1708 - val_accuracy: 0.6139 - val_loss: 1.2963
Epoch 26/40
45/45 0s 195ms/step -
accuracy: 0.6524 - loss: 1.2287
Epoch 26: val_loss did not improve from 1.24474
45/45 9s 205ms/step -
accuracy: 0.6611 - loss: 1.1933 - val_accuracy: 0.6250 - val_loss: 1.2501
Epoch 27/40
45/45 0s 197ms/step -

```

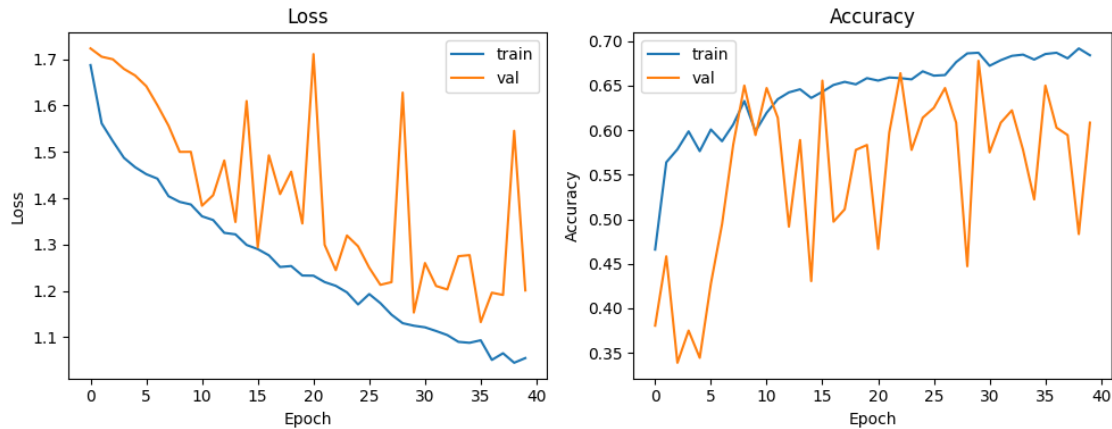
accuracy: 0.6940 - loss: 1.1384
Epoch 27: val_loss improved from 1.24474 to 1.21321, saving model to
best_improved_aug.keras
45/45          9s 208ms/step -
accuracy: 0.6618 - loss: 1.1737 - val_accuracy: 0.6472 - val_loss: 1.2132
Epoch 28/40
45/45          0s 194ms/step -
accuracy: 0.6616 - loss: 1.1677
Epoch 28: val_loss did not improve from 1.21321
45/45          9s 204ms/step -
accuracy: 0.6764 - loss: 1.1487 - val_accuracy: 0.6083 - val_loss: 1.2190
Epoch 29/40
45/45          0s 195ms/step -
accuracy: 0.6964 - loss: 1.1181
Epoch 29: val_loss did not improve from 1.21321
45/45          9s 204ms/step -
accuracy: 0.6861 - loss: 1.1304 - val_accuracy: 0.4472 - val_loss: 1.6280
Epoch 30/40
45/45          0s 194ms/step -
accuracy: 0.6887 - loss: 1.1292
Epoch 30: val_loss improved from 1.21321 to 1.15341, saving model to
best_improved_aug.keras
45/45          9s 205ms/step -
accuracy: 0.6868 - loss: 1.1250 - val_accuracy: 0.6778 - val_loss: 1.1534
Epoch 31/40
45/45          0s 196ms/step -
accuracy: 0.6825 - loss: 1.1080
Epoch 31: val_loss did not improve from 1.15341
45/45          9s 206ms/step -
accuracy: 0.6722 - loss: 1.1215 - val_accuracy: 0.5750 - val_loss: 1.2602
Epoch 32/40
45/45          0s 194ms/step -
accuracy: 0.6778 - loss: 1.1062
Epoch 32: val_loss did not improve from 1.15341
45/45          9s 204ms/step -
accuracy: 0.6785 - loss: 1.1135 - val_accuracy: 0.6083 - val_loss: 1.2107
Epoch 33/40
45/45          0s 195ms/step -
accuracy: 0.6771 - loss: 1.1256
Epoch 33: val_loss did not improve from 1.15341
45/45          9s 205ms/step -
accuracy: 0.6833 - loss: 1.1048 - val_accuracy: 0.6222 - val_loss: 1.2032
Epoch 34/40
45/45          0s 194ms/step -
accuracy: 0.6765 - loss: 1.0970
Epoch 34: val_loss did not improve from 1.15341
45/45          9s 204ms/step -
accuracy: 0.6847 - loss: 1.0901 - val_accuracy: 0.5778 - val_loss: 1.2749

```

```

Epoch 35/40
45/45          0s 196ms/step -
accuracy: 0.6885 - loss: 1.0598
Epoch 35: val_loss did not improve from 1.15341
45/45          9s 205ms/step -
accuracy: 0.6792 - loss: 1.0881 - val_accuracy: 0.5222 - val_loss: 1.2773
Epoch 36/40
45/45          0s 194ms/step -
accuracy: 0.6955 - loss: 1.0616
Epoch 36: val_loss improved from 1.15341 to 1.13274, saving model to
best_improved_aug.keras
45/45          9s 204ms/step -
accuracy: 0.6854 - loss: 1.0935 - val_accuracy: 0.6500 - val_loss: 1.1327
Epoch 37/40
45/45          0s 194ms/step -
accuracy: 0.6821 - loss: 1.0494
Epoch 37: val_loss did not improve from 1.13274
45/45          9s 204ms/step -
accuracy: 0.6868 - loss: 1.0511 - val_accuracy: 0.6028 - val_loss: 1.1962
Epoch 38/40
45/45          0s 194ms/step -
accuracy: 0.6898 - loss: 1.0425
Epoch 38: val_loss did not improve from 1.13274
45/45          9s 204ms/step -
accuracy: 0.6806 - loss: 1.0655 - val_accuracy: 0.5944 - val_loss: 1.1913
Epoch 39/40
45/45          0s 194ms/step -
accuracy: 0.6775 - loss: 1.0722
Epoch 39: val_loss did not improve from 1.13274
45/45          9s 203ms/step -
accuracy: 0.6917 - loss: 1.0450 - val_accuracy: 0.4833 - val_loss: 1.5455
Epoch 40/40
45/45          0s 193ms/step -
accuracy: 0.6649 - loss: 1.0667
Epoch 40: val_loss did not improve from 1.13274
45/45          9s 203ms/step -
accuracy: 0.6840 - loss: 1.0551 - val_accuracy: 0.6083 - val_loss: 1.2011
Restoring model weights from the end of the best epoch: 36.

```



Improved CNN Test accuracy: 0.65

```
[31]: print("=== RESULTS WITH AUGMENTATION ===")
      print(f"Baseline    : {round(acc_baseline, 3)}")
      print(f"Deeper CNN  : {round(acc_deeper, 3)}")
      print(f"Improved CNN: {round(acc_improved, 3)}")
```

=== RESULTS WITH AUGMENTATION ===

Baseline : 0.711

Deeper CNN : 0.706

Improved CNN: 0.65

On remarque que deeper CNN montre le meilleur resultat on terme de graphes

11.3 Dernier modele avec ImageDataGenerator

```
[44]: my_path = "Data_Project/images/"
      my_classes = ['elephant', 'cat', 'sheep']

      X, y = create_X_y(my_path, my_classes)

      X = X.astype("float32") / 255.0
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, stratify=y, random_state=42
      )

      datagen = ImageDataGenerator(
          width_shift_range=0.05,
          height_shift_range=0.05,
          zoom_range=0.1,
          horizontal_flip=True
      )
```

```

train_generator = datagen.flow(
    X_train, y_train,
    batch_size=32,
    shuffle=True
)

def build_final_model(image_size):
    input = Input(shape=(image_size, image_size, 3), name="Input_layer")

    x = Conv2D(32, (3, 3), activation="relu", padding="same")(input)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(64, (3, 3), activation="relu", padding="same")(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(128, (3, 3), activation="relu", padding="same")(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = GlobalAveragePooling2D()(x)

    x = Dense(64, activation="relu",
              kernel_regularizer=regularizers.l2(0.001))(x)
    x = Dropout(0.3)(x)

    output = Dense(3, activation="softmax")(x)

    model = Model(inputs=input, outputs=output, name="Deeper_CNN_FIXED")

    opt = SGD(learning_rate=0.001, momentum=0.9)

    model.compile(
        optimizer=opt,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    return model

image_size = 128
model = build_final_model(image_size)

callbacks = [
    EarlyStopping(

```

```

        monitor='val_loss',
        patience=10,
        restore_best_weights=True,
        verbose=1
    ),
    ModelCheckpoint(
        "best_model.keras",
        monitor='val_loss',
        save_best_only=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        min_lr=1e-5,
        verbose=1
    )
]

history = model.fit(
    train_generator,
    epochs=50,
    validation_data=(X_test, y_test),
    callbacks=callbacks,
    verbose=1
)

plot_history_simple(history)

loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Test accuracy:", round(acc, 3))

y_pred = np.argmax(model.predict(X_test), axis=1)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=my_classes))

```

```

Epoch 1/50
45/45          0s 233ms/step -
accuracy: 0.4551 - loss: 1.3535
Epoch 1: val_loss improved from None to 1.18350, saving model to
best_model.keras
45/45          12s 250ms/step -
accuracy: 0.5132 - loss: 1.1650 - val_accuracy: 0.3333 - val_loss: 1.1835 -
learning_rate: 0.0010

```


Epoch 2/50
 45/45 0s 234ms/step -
 accuracy: 0.6182 - loss: 0.9946
 Epoch 2: val_loss did not improve from 1.18350
 45/45 11s 246ms/step -
 accuracy: 0.6090 - loss: 0.9760 - val_accuracy: 0.3333 - val_loss: 1.1874 -
 learning_rate: 0.0010
 Epoch 3/50
 45/45 0s 233ms/step -
 accuracy: 0.6167 - loss: 0.9352
 Epoch 3: val_loss improved from 1.18350 to 1.16917, saving model to
 best_model.keras
 45/45 11s 246ms/step -
 accuracy: 0.6243 - loss: 0.9274 - val_accuracy: 0.3333 - val_loss: 1.1692 -
 learning_rate: 0.0010
 Epoch 4/50
 45/45 0s 238ms/step -
 accuracy: 0.6298 - loss: 0.9145
 Epoch 4: val_loss improved from 1.16917 to 1.15162, saving model to
 best_model.keras
 45/45 11s 251ms/step -
 accuracy: 0.6410 - loss: 0.8977 - val_accuracy: 0.3972 - val_loss: 1.1516 -
 learning_rate: 0.0010
 Epoch 5/50
 45/45 0s 236ms/step -
 accuracy: 0.6544 - loss: 0.8964
 Epoch 5: val_loss improved from 1.15162 to 1.07107, saving model to
 best_model.keras
 45/45 11s 249ms/step -
 accuracy: 0.6472 - loss: 0.8927 - val_accuracy: 0.4806 - val_loss: 1.0711 -
 learning_rate: 0.0010
 Epoch 6/50
 45/45 0s 240ms/step -
 accuracy: 0.6467 - loss: 0.9020
 Epoch 6: val_loss did not improve from 1.07107
 45/45 11s 252ms/step -
 accuracy: 0.6465 - loss: 0.8843 - val_accuracy: 0.4694 - val_loss: 1.0944 -
 learning_rate: 0.0010
 Epoch 7/50
 45/45 0s 240ms/step -
 accuracy: 0.6528 - loss: 0.8435
 Epoch 7: val_loss improved from 1.07107 to 1.02539, saving model to
 best_model.keras
 45/45 11s 253ms/step -
 accuracy: 0.6618 - loss: 0.8401 - val_accuracy: 0.5278 - val_loss: 1.0254 -
 learning_rate: 0.0010
 Epoch 8/50
 45/45 0s 242ms/step -

```

accuracy: 0.6508 - loss: 0.8621
Epoch 8: val_loss did not improve from 1.02539
45/45          12s 255ms/step -
accuracy: 0.6528 - loss: 0.8533 - val_accuracy: 0.4306 - val_loss: 1.1145 -
learning_rate: 0.0010
Epoch 9/50
45/45          0s 244ms/step -
accuracy: 0.6701 - loss: 0.8560
Epoch 9: val_loss improved from 1.02539 to 0.97707, saving model to
best_model.keras
45/45          12s 257ms/step -
accuracy: 0.6819 - loss: 0.8388 - val_accuracy: 0.5528 - val_loss: 0.9771 -
learning_rate: 0.0010
Epoch 10/50
45/45          0s 235ms/step -
accuracy: 0.6975 - loss: 0.7748
Epoch 10: val_loss improved from 0.97707 to 0.88468, saving model to
best_model.keras
45/45          11s 248ms/step -
accuracy: 0.6847 - loss: 0.7987 - val_accuracy: 0.6250 - val_loss: 0.8847 -
learning_rate: 0.0010
Epoch 11/50
45/45          0s 243ms/step -
accuracy: 0.7115 - loss: 0.7691
Epoch 11: val_loss did not improve from 0.88468
45/45          12s 256ms/step -
accuracy: 0.6993 - loss: 0.7901 - val_accuracy: 0.5806 - val_loss: 0.9460 -
learning_rate: 0.0010
Epoch 12/50
45/45          0s 240ms/step -
accuracy: 0.6931 - loss: 0.7863
Epoch 12: val_loss did not improve from 0.88468
45/45          11s 253ms/step -
accuracy: 0.6896 - loss: 0.8050 - val_accuracy: 0.5583 - val_loss: 1.0408 -
learning_rate: 0.0010
Epoch 13/50
45/45          0s 249ms/step -
accuracy: 0.7245 - loss: 0.7721
Epoch 13: val_loss did not improve from 0.88468

Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
45/45          12s 263ms/step -
accuracy: 0.7083 - loss: 0.7850 - val_accuracy: 0.5944 - val_loss: 0.9350 -
learning_rate: 0.0010
Epoch 14/50
45/45          0s 242ms/step -
accuracy: 0.7042 - loss: 0.7727
Epoch 14: val_loss did not improve from 0.88468

```

45/45 12s 255ms/step -
accuracy: 0.7042 - loss: 0.7792 - val_accuracy: 0.5722 - val_loss: 1.0291 -
learning_rate: 5.0000e-04
Epoch 15/50
45/45 0s 254ms/step -
accuracy: 0.6845 - loss: 0.7710
Epoch 15: val_loss improved from 0.88468 to 0.85571, saving model to
best_model.keras
45/45 12s 268ms/step -
accuracy: 0.7014 - loss: 0.7632 - val_accuracy: 0.6667 - val_loss: 0.8557 -
learning_rate: 5.0000e-04
Epoch 16/50
45/45 0s 244ms/step -
accuracy: 0.7155 - loss: 0.7948
Epoch 16: val_loss improved from 0.85571 to 0.79420, saving model to
best_model.keras
45/45 12s 258ms/step -
accuracy: 0.7160 - loss: 0.7791 - val_accuracy: 0.6972 - val_loss: 0.7942 -
learning_rate: 5.0000e-04
Epoch 17/50
45/45 0s 238ms/step -
accuracy: 0.7230 - loss: 0.7429
Epoch 17: val_loss did not improve from 0.79420
45/45 11s 251ms/step -
accuracy: 0.7160 - loss: 0.7494 - val_accuracy: 0.6444 - val_loss: 0.9597 -
learning_rate: 5.0000e-04
Epoch 18/50
45/45 0s 237ms/step -
accuracy: 0.7344 - loss: 0.7237
Epoch 18: val_loss did not improve from 0.79420
45/45 11s 250ms/step -
accuracy: 0.7174 - loss: 0.7442 - val_accuracy: 0.4972 - val_loss: 1.1910 -
learning_rate: 5.0000e-04
Epoch 19/50
45/45 0s 240ms/step -
accuracy: 0.6967 - loss: 0.7957
Epoch 19: val_loss improved from 0.79420 to 0.78381, saving model to
best_model.keras
45/45 12s 254ms/step -
accuracy: 0.7049 - loss: 0.7740 - val_accuracy: 0.7167 - val_loss: 0.7838 -
learning_rate: 5.0000e-04
Epoch 20/50
45/45 0s 240ms/step -
accuracy: 0.7308 - loss: 0.7172
Epoch 20: val_loss did not improve from 0.78381
45/45 11s 253ms/step -
accuracy: 0.7146 - loss: 0.7423 - val_accuracy: 0.6778 - val_loss: 0.8854 -
learning_rate: 5.0000e-04

Epoch 21/50
 45/45 0s 242ms/step -
 accuracy: 0.7195 - loss: 0.7137
 Epoch 21: val_loss did not improve from 0.78381
 45/45 12s 256ms/step -
 accuracy: 0.7194 - loss: 0.7339 - val_accuracy: 0.6833 - val_loss: 0.8405 -
 learning_rate: 5.0000e-04
 Epoch 22/50
 45/45 0s 245ms/step -
 accuracy: 0.7254 - loss: 0.7365
 Epoch 22: val_loss did not improve from 0.78381

Epoch 22: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
 45/45 12s 258ms/step -
 accuracy: 0.7188 - loss: 0.7396 - val_accuracy: 0.7000 - val_loss: 0.8031 -
 learning_rate: 5.0000e-04
 Epoch 23/50
 45/45 0s 246ms/step -
 accuracy: 0.7317 - loss: 0.7212
 Epoch 23: val_loss did not improve from 0.78381
 45/45 12s 260ms/step -
 accuracy: 0.7271 - loss: 0.7220 - val_accuracy: 0.7139 - val_loss: 0.8057 -
 learning_rate: 2.5000e-04
 Epoch 24/50
 45/45 0s 248ms/step -
 accuracy: 0.7482 - loss: 0.7179
 Epoch 24: val_loss improved from 0.78381 to 0.77429, saving model to
 best_model.keras
 45/45 12s 261ms/step -
 accuracy: 0.7285 - loss: 0.7441 - val_accuracy: 0.7389 - val_loss: 0.7743 -
 learning_rate: 2.5000e-04
 Epoch 25/50
 45/45 0s 247ms/step -
 accuracy: 0.7493 - loss: 0.6982
 Epoch 25: val_loss did not improve from 0.77429
 45/45 12s 260ms/step -
 accuracy: 0.7417 - loss: 0.7079 - val_accuracy: 0.6000 - val_loss: 1.0380 -
 learning_rate: 2.5000e-04
 Epoch 26/50
 45/45 0s 249ms/step -
 accuracy: 0.7338 - loss: 0.7246
 Epoch 26: val_loss did not improve from 0.77429
 45/45 12s 262ms/step -
 accuracy: 0.7465 - loss: 0.7089 - val_accuracy: 0.6361 - val_loss: 0.9229 -
 learning_rate: 2.5000e-04
 Epoch 27/50
 45/45 0s 249ms/step -
 accuracy: 0.7400 - loss: 0.6895

Epoch 27: val_loss did not improve from 0.77429

Epoch 27: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

45/45 12s 263ms/step -

accuracy: 0.7396 - loss: 0.7150 - val_accuracy: 0.6694 - val_loss: 0.8756 -
learning_rate: 2.5000e-04

Epoch 28/50

45/45 0s 249ms/step -

accuracy: 0.7473 - loss: 0.6933

Epoch 28: val_loss did not improve from 0.77429

45/45 12s 262ms/step -

accuracy: 0.7340 - loss: 0.7032 - val_accuracy: 0.6972 - val_loss: 0.8260 -
learning_rate: 1.2500e-04

Epoch 29/50

45/45 0s 249ms/step -

accuracy: 0.7338 - loss: 0.6892

Epoch 29: val_loss did not improve from 0.77429

45/45 12s 262ms/step -

accuracy: 0.7368 - loss: 0.6989 - val_accuracy: 0.6722 - val_loss: 0.8573 -
learning_rate: 1.2500e-04

Epoch 30/50

45/45 0s 252ms/step -

accuracy: 0.7367 - loss: 0.7097

Epoch 30: val_loss did not improve from 0.77429

Epoch 30: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

45/45 12s 265ms/step -

accuracy: 0.7271 - loss: 0.7112 - val_accuracy: 0.6000 - val_loss: 0.9814 -
learning_rate: 1.2500e-04

Epoch 31/50

45/45 0s 251ms/step -

accuracy: 0.7739 - loss: 0.6743

Epoch 31: val_loss did not improve from 0.77429

45/45 12s 265ms/step -

accuracy: 0.7632 - loss: 0.6758 - val_accuracy: 0.6694 - val_loss: 0.8504 -
learning_rate: 6.2500e-05

Epoch 32/50

45/45 0s 250ms/step -

accuracy: 0.7611 - loss: 0.6670

Epoch 32: val_loss did not improve from 0.77429

45/45 12s 264ms/step -

accuracy: 0.7549 - loss: 0.6840 - val_accuracy: 0.7250 - val_loss: 0.7786 -
learning_rate: 6.2500e-05

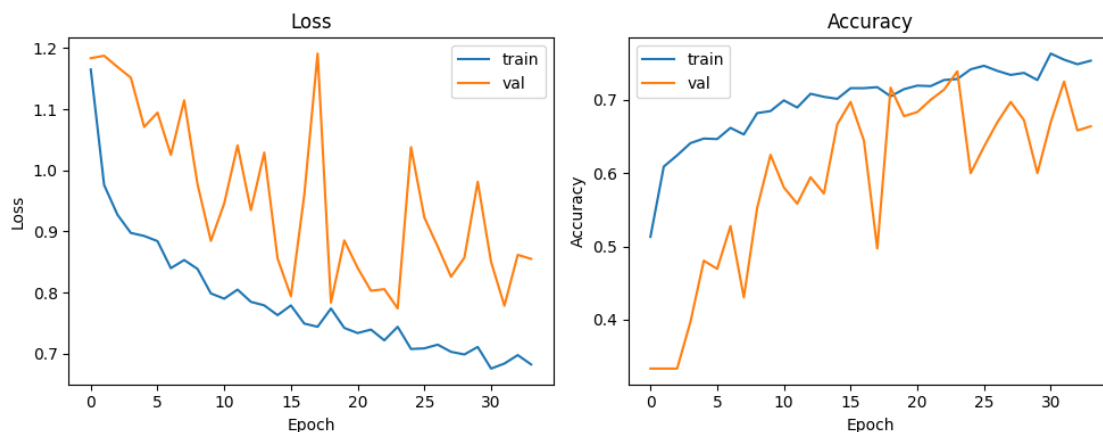
Epoch 33/50

45/45 0s 252ms/step -

accuracy: 0.7294 - loss: 0.7161

Epoch 33: val_loss did not improve from 0.77429

Epoch 33: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
 45/45 12s 266ms/step -
 accuracy: 0.7486 - loss: 0.6979 - val_accuracy: 0.6583 - val_loss: 0.8619 -
 learning_rate: 6.2500e-05
 Epoch 34/50
 45/45 0s 251ms/step -
 accuracy: 0.7452 - loss: 0.6967
 Epoch 34: val_loss did not improve from 0.77429
 45/45 12s 264ms/step -
 accuracy: 0.7535 - loss: 0.6826 - val_accuracy: 0.6639 - val_loss: 0.8552 -
 learning_rate: 3.1250e-05
 Epoch 34: early stopping
 Restoring model weights from the end of the best epoch: 24.



Test accuracy: 0.739

WARNING:tensorflow:5 out of the last 15 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7cc9323860e0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.
 12/12 1s 55ms/step

Classification Report:

	precision	recall	f1-score	support
elephant	0.68	0.78	0.73	120

cat	0.89	0.72	0.79	120
sheep	0.69	0.72	0.70	120
accuracy			0.74	360
macro avg	0.75	0.74	0.74	360
weighted avg	0.75	0.74	0.74	360

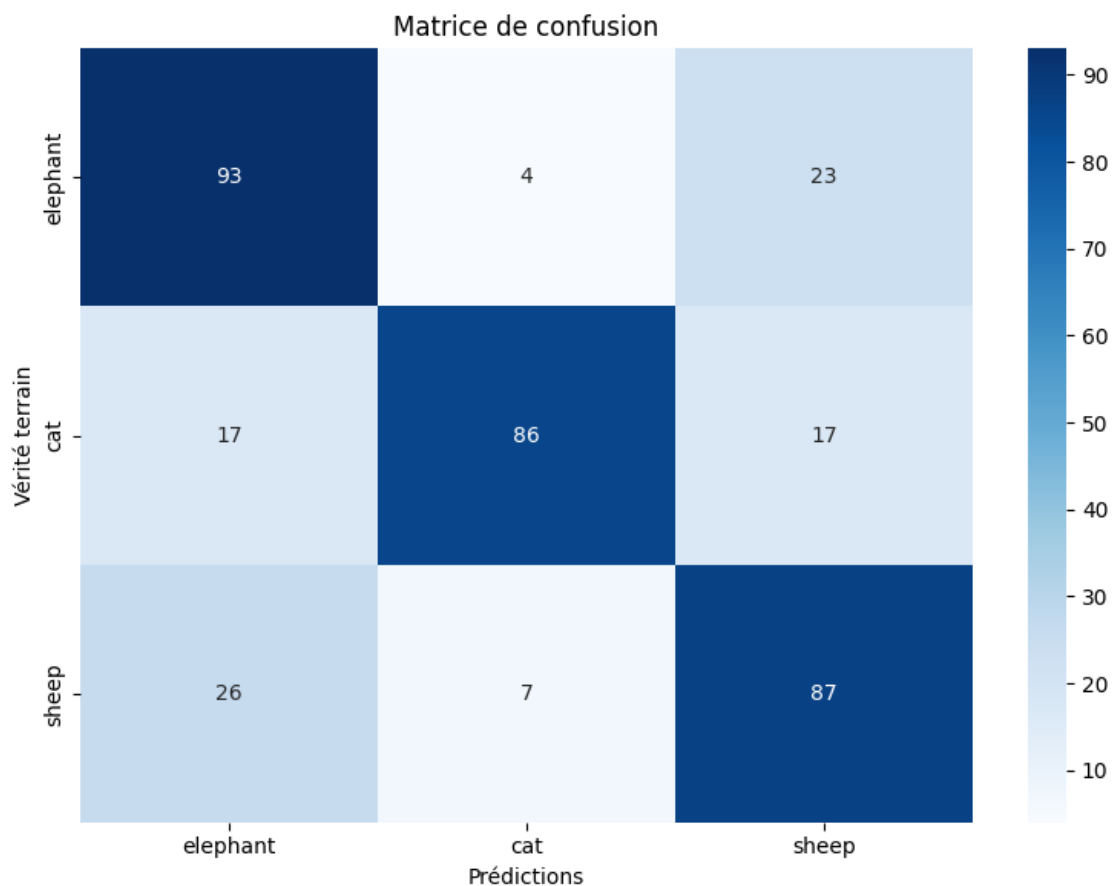
```
[45]: y_pred_labels = np.argmax(model.predict(X_test), axis=1)

conf = confusion_matrix(y_test, y_pred_labels)

plt.figure(figsize=(8, 6))
sns.heatmap(conf, annot=True, fmt='d', cmap='Blues',
            xticklabels=my_classes, yticklabels=my_classes)
plt.title('Matrice de confusion')
plt.xlabel('Prédictions')
plt.ylabel('Vérité terrain')
plt.tight_layout()
plt.show()
```

12/12

1s 47ms/step



11.4 Feature Maps

```
[54]: idx=23
img = X[idx]

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6,6))
plt.imshow(img_rgb)
plt.axis("off")
plt.show()
```



```
[55]: for layer in model.layers:
        print(layer.name, layer.output.shape)
```

Input_layer (None, 128, 128, 3)


```

conv2d_18 (None, 128, 128, 32)
batch_normalization_6 (None, 128, 128, 32)
max_pooling2d_18 (None, 64, 64, 32)
conv2d_19 (None, 64, 64, 64)
batch_normalization_7 (None, 64, 64, 64)
max_pooling2d_19 (None, 32, 32, 64)
conv2d_20 (None, 32, 32, 128)
batch_normalization_8 (None, 32, 32, 128)
max_pooling2d_20 (None, 16, 16, 128)
global_average_pooling2d_6 (None, 128)
dense_12 (None, 64)
dropout_12 (None, 64)
dense_13 (None, 3)

```

```

[56]: list_layer_conv = [l.name for l in model.layers if 'conv2d' in l.name]
      print("Couches Conv détectées :", list_layer_conv)

      outputs = [model.get_layer(name).output for name in list_layer_conv]
      feature_maps_model = tf.keras.models.Model(inputs=model.input, outputs=outputs)
      feature_maps = feature_maps_model.predict(my_image)

      for layer_name, fmap in zip(list_layer_conv, feature_maps):
          print(f"\n{layer_name}'s shape : {fmap.shape}")

          num_maps = fmap.shape[-1]
          cols = 8
          rows = (num_maps // cols) + (num_maps % cols > 0)

          fig = plt.figure(figsize=(20, rows * 2))
          fig.suptitle(f"Feature Maps : {layer_name}", fontsize=14)

          for j in range(num_maps):
              plt.subplot(rows, cols, j + 1)
              plt.imshow(fmap[0, :, :, j], cmap='viridis')
              plt.axis('off')

          plt.tight_layout()
          plt.show()

```

```

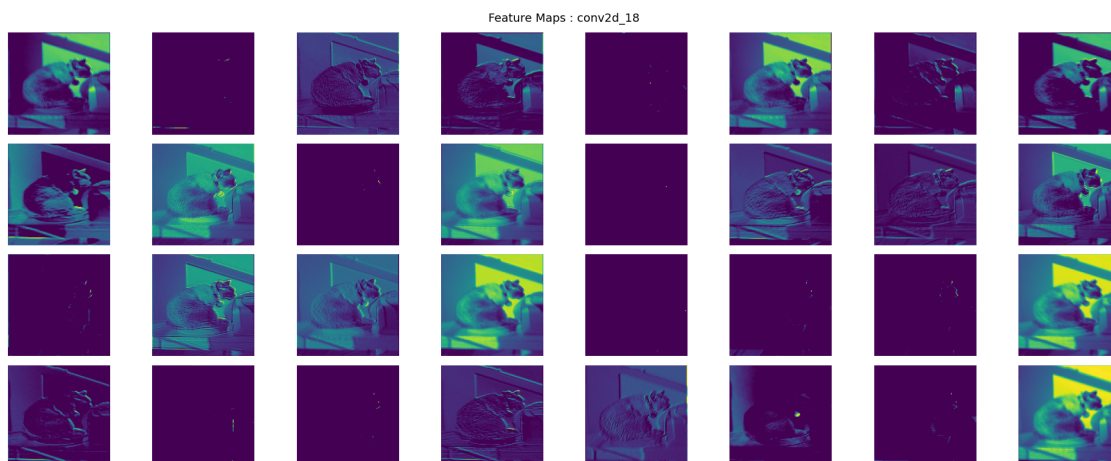
Couches Conv détectées : ['conv2d_18', 'conv2d_19', 'conv2d_20']
1/1          0s 57ms/step

```

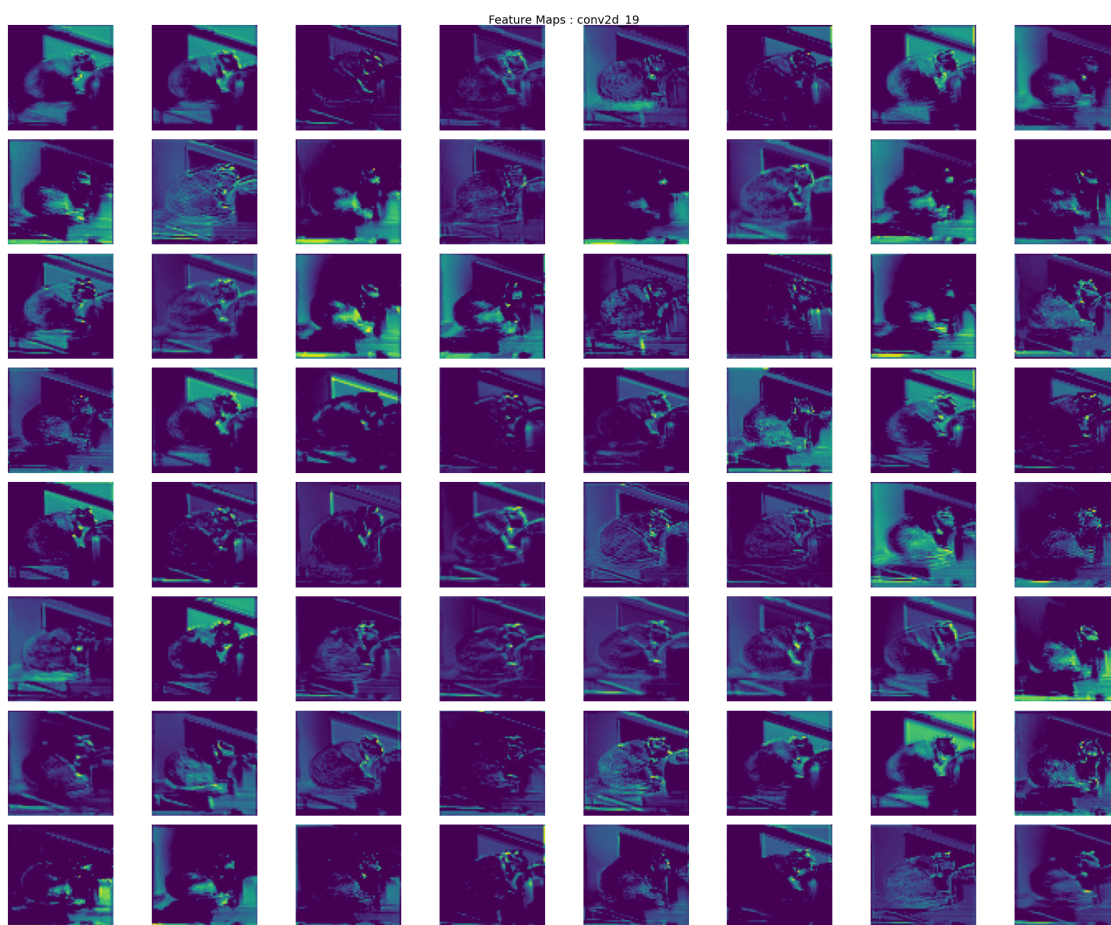
```

conv2d_18's shape : (1, 128, 128, 32)

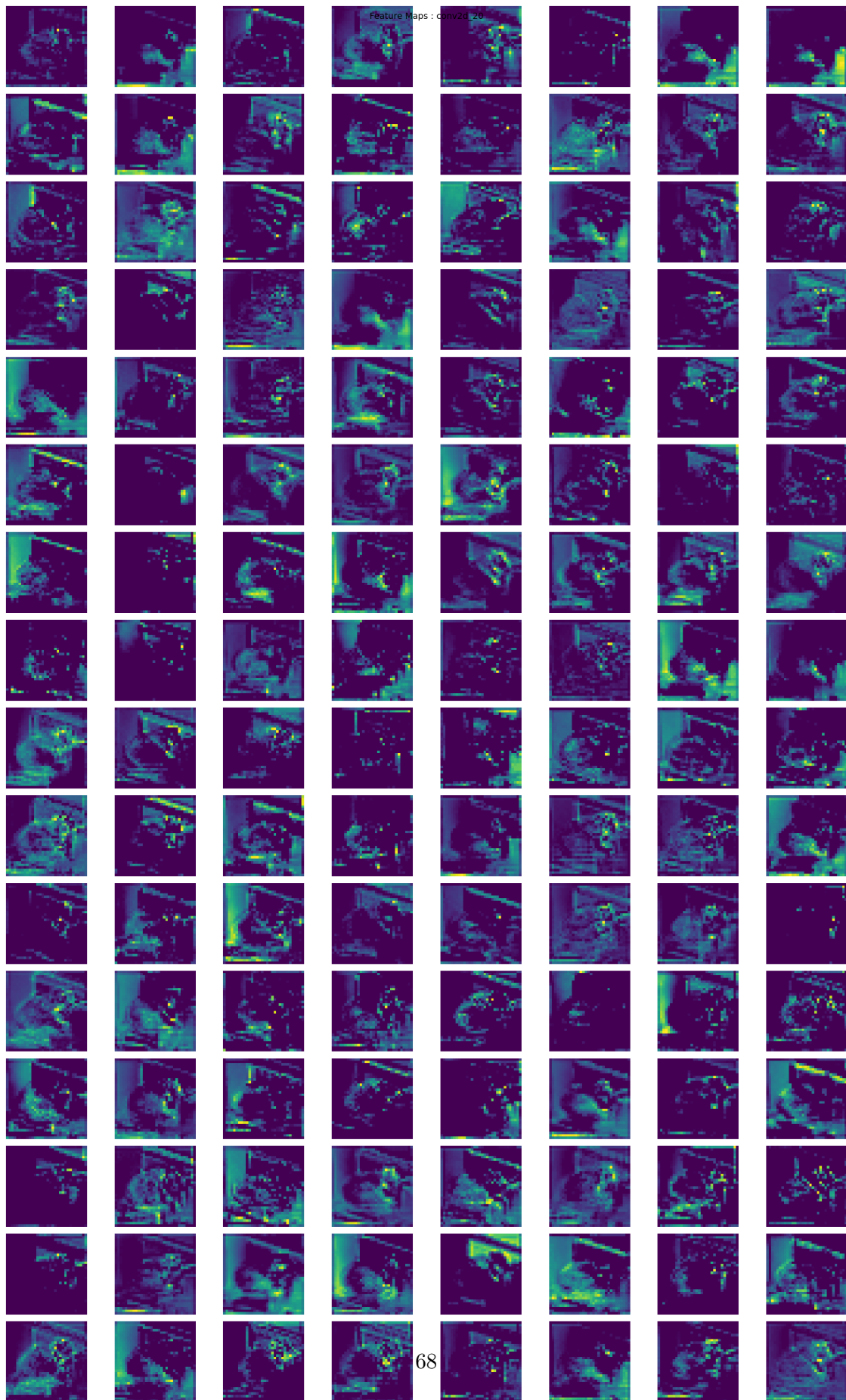
```



conv2d_19's shape : (1, 64, 64, 64)



```
conv2d_20's shape : (1, 32, 32, 128)
```



12 Transfer Learning

```
[19]: def transfer_model(image_size=128, num_classes=3):
    inputs = Input(shape=(image_size, image_size, 3), name="Input_Layer")

    x = preprocess_input(inputs * 255.0)

    base_model = ResNet50(
        include_top=False,
        pooling='avg',
        weights='imagenet',
        input_shape=(image_size, image_size, 3)
    )
    base_model.trainable = False

    x = base_model(x)
    x = Dense(512, activation='relu', name="Dense1_Layer")(x)
    x = Dropout(0.3)(x)
    outputs = Dense(num_classes, activation='softmax', name="Output_Layer")(x)

    model = Model(inputs=inputs, outputs=outputs, name="ResNet50_Transfer")
    model.compile(optimizer=Adam(learning_rate=0.001),
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    return model
```

```
[20]: transfer_model().summary()
```

Model: "ResNet50_Transfer"

Layer (type)	Output Shape	Param #	Connected to
Input_Layer (InputLayer)	(None, 128, 128, 3)	0	-
multiply (Multiply)	(None, 128, 128, 3)	0	Input_Layer[0][0]
get_item (GetItem)	(None, 128, 128)	0	multiply[0][0]
get_item_1 (GetItem)	(None, 128, 128)	0	multiply[0][0]

get_item_2 (GetItem)	(None, 128, 128)	0	multiply[0][0]
stack (Stack)	(None, 128, 128, 3)	0	get_item[0][0], get_item_1[0][0], get_item_2[0][0]
add (Add)	(None, 128, 128, 3)	0	stack[0][0]
resnet50 (Functional)	(None, 2048)	23,587,712	add[0][0]
Dense1_Layer (Dense)	(None, 512)	1,049,088	resnet50[0][0]
dropout_1 (Dropout)	(None, 512)	0	Dense1_Layer[0][...]
Output_Layer (Dense)	(None, 3)	1,539	dropout_1[0][0]

Total params: 24,638,339 (93.99 MB)

Trainable params: 1,050,627 (4.01 MB)

Non-trainable params: 23,587,712 (89.98 MB)

```
[21]: my_path = "Data_Project/images/"
my_classes = ['elephant', 'cat', 'sheep']

X, y = create_X_y(my_path, my_classes)

X = X.astype("float32") / 255.0
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

```
[22]: datagen = ImageDataGenerator(
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True
)
```

```

train_generator = datagen.flow(X_train, y_train, batch_size=32, shuffle=True)

model_transfer, history_transfer = train_cnn_model(
    transfer_model(),
    train_generator=train_generator,
    val_data=(X_test, y_test),
    model_path="best_transfer.keras",
    epochs=20,
    patience=5
)
plot_history_simple(history_transfer)
_, acc_transfer = model_transfer.evaluate(X_test, y_test, verbose=0)
print("Transfer Learning Test accuracy:", round(acc_transfer, 3))

```

Epoch 1/20

45/45 0s 242ms/step -

accuracy: 0.6342 - loss: 2.3228

Epoch 1: val_loss improved from None to 0.36351, saving model to
best_transfer.keras

45/45 18s 335ms/step -

accuracy: 0.7528 - loss: 1.1403 - val_accuracy: 0.8528 - val_loss: 0.3635

Epoch 2/20

45/45 0s 246ms/step -

accuracy: 0.8447 - loss: 0.3428

Epoch 2: val_loss did not improve from 0.36351

45/45 14s 309ms/step -

accuracy: 0.8611 - loss: 0.3338 - val_accuracy: 0.8583 - val_loss: 0.4111

Epoch 3/20

45/45 0s 256ms/step -

accuracy: 0.8858 - loss: 0.2953

Epoch 3: val_loss improved from 0.36351 to 0.29598, saving model to
best_transfer.keras

45/45 15s 333ms/step -

accuracy: 0.8861 - loss: 0.2881 - val_accuracy: 0.8694 - val_loss: 0.2960

Epoch 4/20

45/45 0s 267ms/step -

accuracy: 0.9060 - loss: 0.2381

Epoch 4: val_loss did not improve from 0.29598

45/45 15s 334ms/step -

accuracy: 0.9181 - loss: 0.2180 - val_accuracy: 0.8917 - val_loss: 0.2995

Epoch 5/20

45/45 0s 270ms/step -

accuracy: 0.9379 - loss: 0.1720

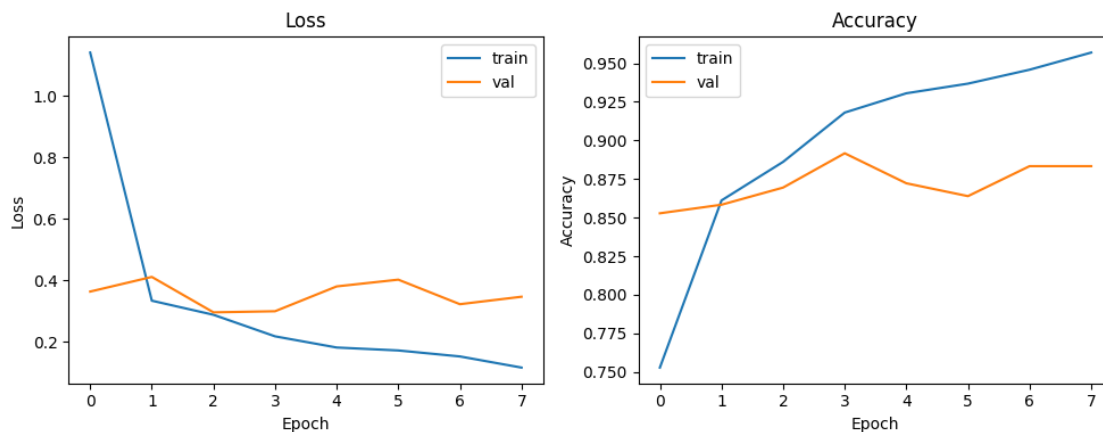
Epoch 5: val_loss did not improve from 0.29598

45/45 15s 338ms/step -

accuracy: 0.9306 - loss: 0.1816 - val_accuracy: 0.8722 - val_loss: 0.3802

Epoch 6/20

45/45 0s 276ms/step -
accuracy: 0.9330 - loss: 0.1827
Epoch 6: val_loss did not improve from 0.29598
45/45 16s 346ms/step -
accuracy: 0.9368 - loss: 0.1721 - val_accuracy: 0.8639 - val_loss: 0.4022
Epoch 7/20
45/45 0s 280ms/step -
accuracy: 0.9539 - loss: 0.1326
Epoch 7: val_loss did not improve from 0.29598
45/45 16s 350ms/step -
accuracy: 0.9458 - loss: 0.1526 - val_accuracy: 0.8833 - val_loss: 0.3224
Epoch 8/20
45/45 0s 282ms/step -
accuracy: 0.9546 - loss: 0.1178
Epoch 8: val_loss did not improve from 0.29598
45/45 16s 353ms/step -
accuracy: 0.9569 - loss: 0.1164 - val_accuracy: 0.8833 - val_loss: 0.3467
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.



Transfer Learning Test accuracy: 0.869

```
[23]: y_pred = np.argmax(model_transfer.predict(X_test), axis=1)
print(classification_report(y_test, y_pred, target_names=my_classes))

plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
             cmap='Blues', xticklabels=my_classes, yticklabels=my_classes)
plt.title("Matrice de confusion")
plt.xlabel("Prédictions")
plt.ylabel("Vérité terrain")
plt.tight_layout()
```

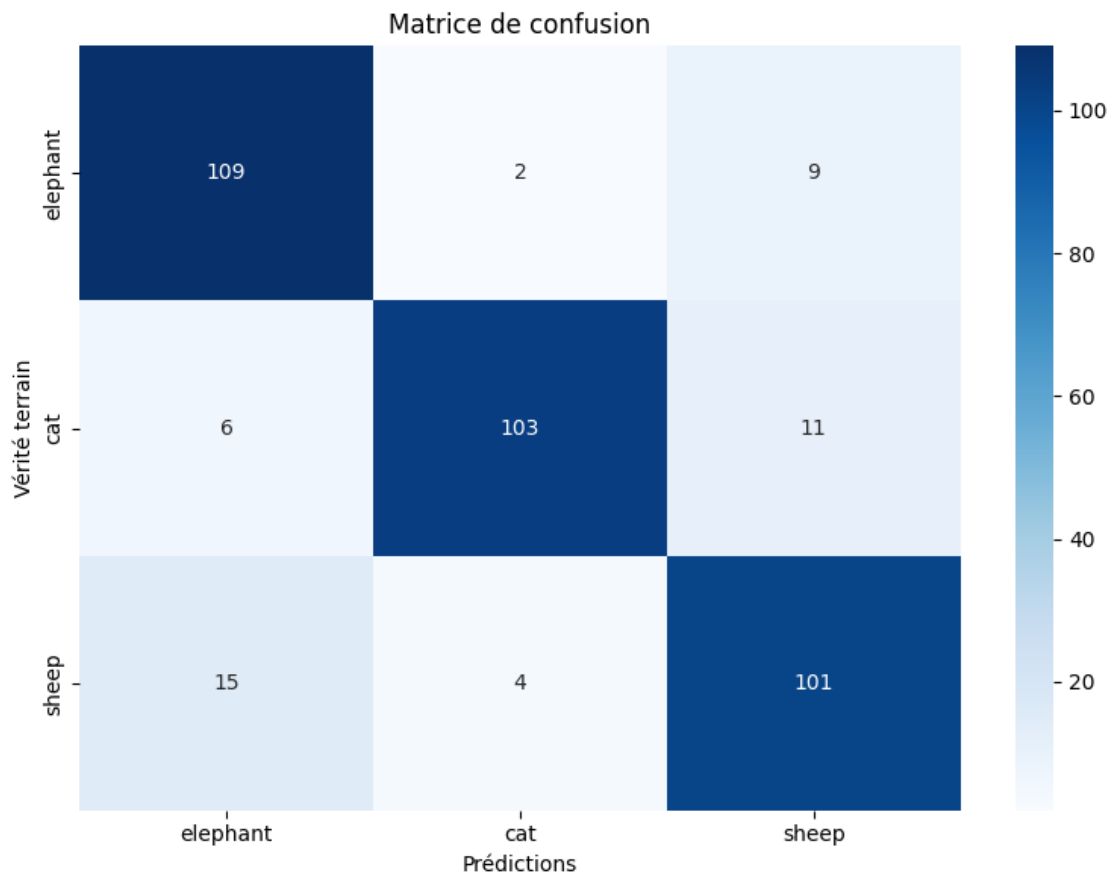


```
plt.show()

print(f"Transfer Learning : {round(acc_transfer, 3)}")
```

12/12 5s 341ms/step

	precision	recall	f1-score	support
elephant	0.84	0.91	0.87	120
cat	0.94	0.86	0.90	120
sheep	0.83	0.84	0.84	120
accuracy			0.87	360
macro avg	0.87	0.87	0.87	360
weighted avg	0.87	0.87	0.87	360



Transfer Learning : 0.869

13 Bismillah we'll unfreeze the other layers

```
[32]: def final_transfer_model(image_size=128, num_classes=3):
    inputs = Input(shape=(image_size, image_size, 3), name="Input_Layer")

    x = preprocess_input(inputs * 255.0)

    base_model = ResNet50(
        include_top=False,
        pooling='avg',
        weights='imagenet',
        input_shape=(image_size, image_size, 3)
    )
    base_model.trainable = True

    x = base_model(x)
    x = Dense(512, activation='relu', name="Dense1_Layer")(x)
    x = Dropout(0.3)(x)
    outputs = Dense(num_classes, activation='softmax', name="Output_Layer")(x)

    model = Model(inputs=inputs, outputs=outputs, name="ResNet50_Finetuned")
    model.compile(optimizer=Adam(learning_rate=1e-5), # very small lr -
    ↪critical
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])

    return model
```

```
[33]: model= final_transfer_model()
model.summary()
```

Model: "ResNet50_Finetuned"

Layer (type)	Output Shape	Param #	Connected to
Input_Layer (InputLayer)	(None, 128, 128, 3)	0	-
multiply_6 (Multiply)	(None, 128, 128, 3)	0	Input_Layer[0][0]
get_item_18 (GetItem)	(None, 128, 128)	0	multiply_6[0][0]
get_item_19 (GetItem)	(None, 128, 128)	0	multiply_6[0][0]

get_item_20 (GetItem)	(None, 128, 128)	0	multiply_6[0][0]
stack_6 (Stack)	(None, 128, 128, 3)	0	get_item_18[0][0... get_item_19[0][0... get_item_20[0][0]
add_6 (Add)	(None, 128, 128, 3)	0	stack_6[0][0]
resnet50 (Functional)	(None, 2048)	23,587,712	add_6[0][0]
Dense1_Layer (Dense)	(None, 512)	1,049,088	resnet50[0][0]
dropout_6 (Dropout)	(None, 512)	0	Dense1_Layer[0][...
Output_Layer (Dense)	(None, 3)	1,539	dropout_6[0][0]

Total params: 24,638,339 (93.99 MB)

Trainable params: 24,585,219 (93.79 MB)

Non-trainable params: 53,120 (207.50 KB)

```
[34]: my_path = "Data_Project/images/"
my_classes = ['elephant', 'cat', 'sheep']

X, y = create_X_y(my_path, my_classes)

X = X.astype("float32") / 255.0
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

```
[35]: datagen = ImageDataGenerator(
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.1,
    horizontal_flip=True
)
train_generator = datagen.flow(X_train, y_train, batch_size=32, shuffle=True)
```

```

model_transfer, history_transfer = train_cnn_model(
    transfer_model(),
    train_generator=train_generator,
    val_data=(X_test, y_test),
    model_path="best_best_transfer.keras",
    epochs=20,
    patience=5
)
plot_history_simple(history_transfer)
_, acc_transfer = model_transfer.evaluate(X_test, y_test, verbose=0)
print("Transfer Learning Test accuracy:", round(acc_transfer, 3))

```

Epoch 1/20

45/45 0s 275ms/step -

accuracy: 0.6492 - loss: 2.1989

Epoch 1: val_loss improved from None to 0.39857, saving model to

best_best_transfer.keras

45/45 20s 382ms/step -

accuracy: 0.7444 - loss: 1.1978 - val_accuracy: 0.8639 - val_loss: 0.3986

Epoch 2/20

45/45 0s 284ms/step -

accuracy: 0.8638 - loss: 0.3083

Epoch 2: val_loss improved from 0.39857 to 0.28782, saving model to

best_best_transfer.keras

45/45 17s 368ms/step -

accuracy: 0.8687 - loss: 0.3037 - val_accuracy: 0.9056 - val_loss: 0.2878

Epoch 3/20

45/45 0s 291ms/step -

accuracy: 0.8861 - loss: 0.2915

Epoch 3: val_loss did not improve from 0.28782

45/45 16s 364ms/step -

accuracy: 0.8986 - loss: 0.2582 - val_accuracy: 0.9028 - val_loss: 0.3036

Epoch 4/20

45/45 0s 294ms/step -

accuracy: 0.9207 - loss: 0.2010

Epoch 4: val_loss did not improve from 0.28782

45/45 17s 368ms/step -

accuracy: 0.9243 - loss: 0.2081 - val_accuracy: 0.8944 - val_loss: 0.3389

Epoch 5/20

45/45 0s 294ms/step -

accuracy: 0.9029 - loss: 0.2268

Epoch 5: val_loss did not improve from 0.28782

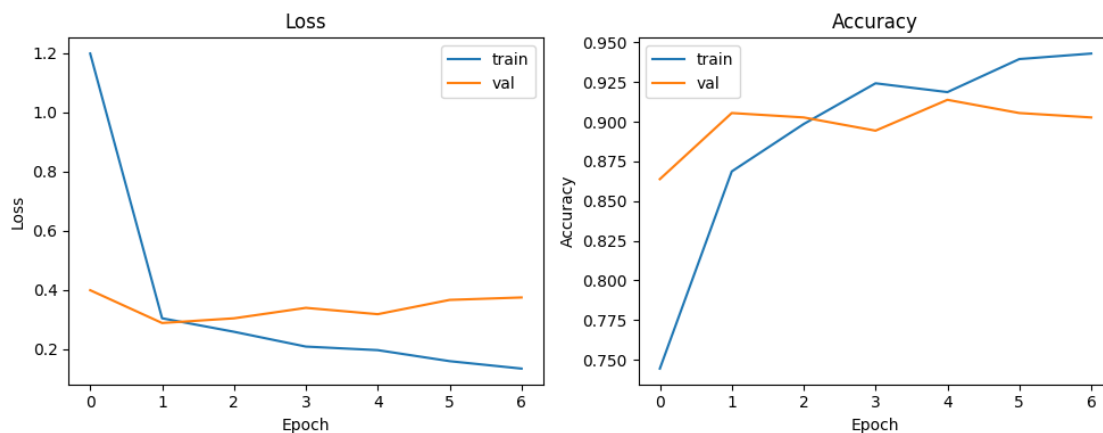
45/45 17s 367ms/step -

accuracy: 0.9187 - loss: 0.1961 - val_accuracy: 0.9139 - val_loss: 0.3176

Epoch 6/20

45/45 0s 295ms/step -

accuracy: 0.9412 - loss: 0.1500
Epoch 6: val_loss did not improve from 0.28782
45/45 17s 370ms/step -
accuracy: 0.9396 - loss: 0.1589 - val_accuracy: 0.9056 - val_loss: 0.3658
Epoch 7/20
45/45 0s 293ms/step -
accuracy: 0.9467 - loss: 0.1264
Epoch 7: val_loss did not improve from 0.28782
45/45 17s 369ms/step -
accuracy: 0.9431 - loss: 0.1338 - val_accuracy: 0.9028 - val_loss: 0.3740
Epoch 7: early stopping
Restoring model weights from the end of the best epoch: 2.



Transfer Learning Test accuracy: 0.906

```
[36]: y_pred = np.argmax(model_transfer.predict(X_test), axis=1)
print(classification_report(y_test, y_pred, target_names=my_classes))

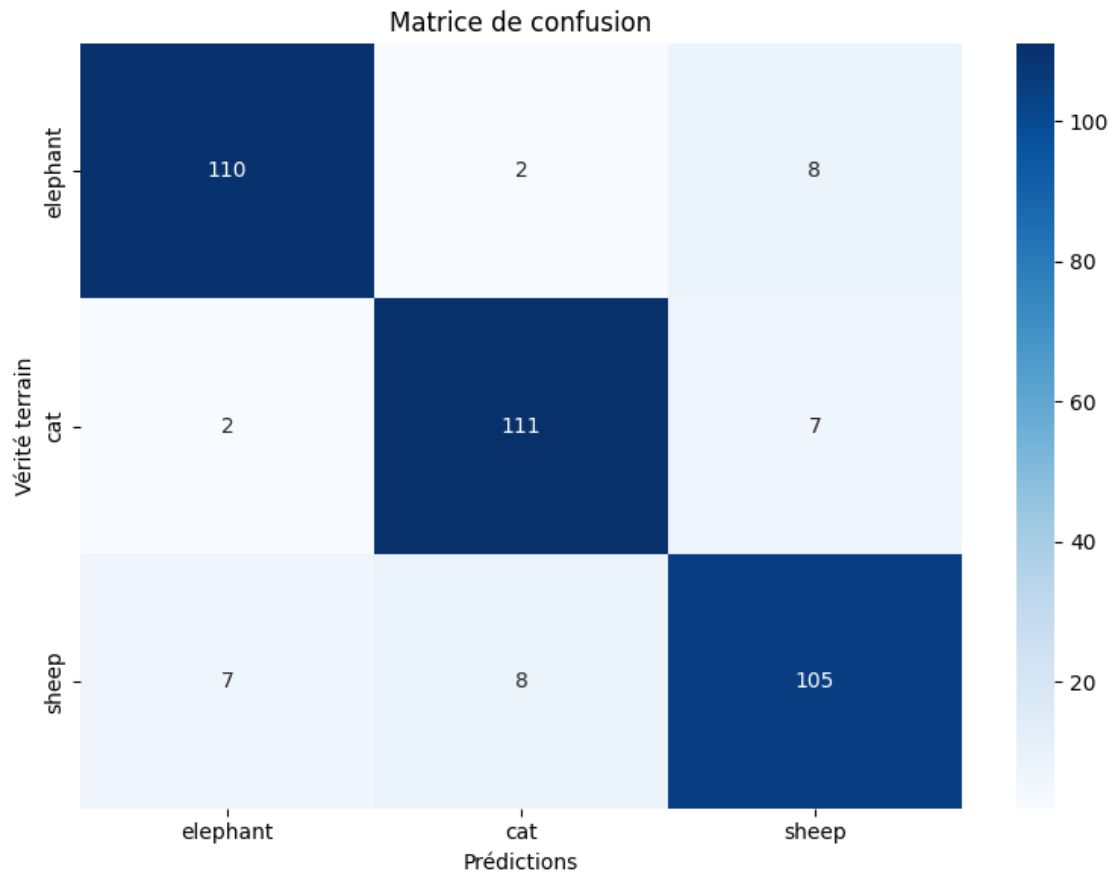
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
            cmap='Blues', xticklabels=my_classes, yticklabels=my_classes)
plt.title("Matrice de confusion")
plt.xlabel("Prédictions")
plt.ylabel("Vérité terrain")
plt.tight_layout()
plt.show()

print(f"Transfer Learning : {round(acc_transfer, 3)}")
```

12/12 5s 374ms/step

	precision	recall	f1-score	support
elephant	0.92	0.92	0.92	120

cat	0.92	0.93	0.92	120
sheep	0.88	0.88	0.88	120
accuracy			0.91	360
macro avg	0.91	0.91	0.91	360
weighted avg	0.91	0.91	0.91	360



Transfer Learning : 0.906